

SAT Encoding for Set of At-Most-K Cardinality Constraints with Ladder Shape

Hieu Truong Xuan^{1†}, Tuyen Kieu Van¹, Khanh To Van^{1*†}

¹University of Engineering and Technology,
Vietnam National University, Hanoi, Vietnam.

*Corresponding author(s). E-mail(s): khanhtv@vnu.edu.vn;
Contributing authors: truongxuanhieu11@vnu.edu.vn;
tuyenkv@vnu.edu.vn;

[†]These authors contributed equally to this work.

Abstract

This paper presents a novel SAT encoding designed to represent a sequence of At-Most-K (AMK) cardinality constraints arranged in a ladder-like structure, referred to as the Ladder AMK constraint. These AMK constraints feature a ladder-like arrangement, with shared common variables in the center and two variables differing at the beginning and the end. The proposed encoding first decomposes the sequence of AMK constraints into separated blocks, then encodes each block as a whole rather than encoding each AMK constraint individually. This approach improves the efficiency of constraint solving in SAT solvers by significantly reducing the size of the encoding, precisely the number of variables, and the generated clauses required. To achieve this, the encoding employs *Sequential Counter* to encode each block and constructs clauses that link adjacent blocks, ensuring equivalence with the input constraints. Compared to the previously proposed encoding that uses *Binary Decision Diagrams* for Ladder AMO constraint ($k = 1$), our encoding requires fewer variables and clauses. Our study is the first study that develops SAT encoding for Ladder AMK constraints with $k > 1$. Experimental results on single Ladder AMO and Ladder AMK constraints indicate that our encoding significantly enhances both the size of the encoding and the efficiency of solving time compared to existing encodings. Furthermore, by evaluating on the two real-world combinatorial problems, namely Anti-bandwidth Problem and Nurse Rostering Problem, we found that the proposed encoding surpasses alternative encodings, as well as more advanced Constraint Programming (CP) and Mixed Integer Programming (MIP) solvers. In summary, the SAT encoding technique presented in this paper provides a more efficient method for encoding sequence constraint, specifically Ladder AMK constraint, which could

enhance the effectiveness of SAT solving for various problems involving this type of constraint.

Keywords: SAT encoding, Cardinality constraints, Sequence constraints, Sequential Counter encoding, At-Most-K constraints, Anti-bandwidth

1 Introduction

Sequence constraint is a type of constraint that frequently appears in combinatorial optimization problems, such as production line scheduling, employee scheduling, Nurse Rostering Problem, Crew Rostering Problem, sports timetabling, and OR problems. For instance, in the Car Sequencing Problem [1, 2], sequence constraints restrict the maximum number of cars with a specific option to no more than two in any sequence of five consecutive cars on a production line. In the Nurse Rostering Problem [3, 4], sequence constraints may limit a nurse to working at most four night shifts and eight evening shifts, while also requiring at least four off-days within any sequence of fourteen consecutive days [5]. These constraints help ensure proper rest and recovery by preventing excessively intense work schedules. Similarly, in the Crew Rostering Problem, an employee may be allowed to work a maximum of five days in any seven consecutive days [6] to comply with labor regulations. In the taxi driver shift scheduling problem at an airport for a car rental company [7], sequence constraints are used to ensure a balanced distribution of morning and afternoon shifts among drivers. In the RobinX sports timetabling problem [8], sequence constraints limit the maximum number of home or away games a team can play within any fixed-length sequence of consecutive matches. Sequence constraints limit the number of occurrences of certain specific values in a sequence of k variables and are often referred to as AmongSeq [9] and AtMostSeq [1]. Certain problems, such as the Anti-bandwidth Problem (ABP) [10], can be reformulated to incorporate sequence constraints. This problem has applications in scheduling, radio frequency assignment, obnoxious facility location, and map coloring.

The impressive progress of SAT solvers has initiated what is now referred to as the SAT revolution [11]. These solvers are capable of addressing enormous formulas that contain millions of variables and clauses. In the near future, they are expected to handle formulas with significantly more than just millions of variables and clauses. Today, SAT solving has become a serious alternative to Integer Linear Programming (ILP) solvers for addressing NP-complete problems. Due to substantial improvements in the performance of SAT solvers, SAT solving is increasingly being used in various real-world applications. However, when applying SAT encoding to combinatorial problems, a significant number of clauses can be generated. To mitigate this issue, several encoding techniques have been proposed to reduce the overwhelming number of clauses [12, 13].

Sequence constraints have been extensively studied in the field of Constraint Programming (CP) to develop efficient solutions, as highlighted in the work of Bergman

et al. [5] and Siala et al. [2]. However, effective SAT encodings for sequence constraints, such as AmongSeq [9] and AtMostSeq [1], remain largely unexamined. A notable exception is *Duplex* encoding [14], which is specifically designed for sequences of At-Most-One (AMO) constraints. In our research, we propose a block-based encoding method for a sequence of At-Most-K (AMK) constraints, which encodes them as a whole rather than handling each AMK constraint independently. In particular, we leverage shared variables among neighboring AMK constraints to decompose the input sequence constraint into blocks. Then, we encode these blocks using an encoding inspired by the *Sequential Counter* [15] encoding. Auxiliary variables generated from the block encoding are used to connect adjacent blocks, thereby reformulating the original constraint. This approach not only demonstrates greater efficiency for sequences of AMO constraints compared to the *Duplex* encoding, as reported by Hieu et al. [16], but also proves to be effective for sequences of AMK constraints with $k > 1$.

- We propose a novel SAT encoding for a set of AMK cardinality constraints arranged in a ladder shape. Our approach leverages shared variables among neighboring AMK constraints, allowing us to organize them into blocks for encoding rather than encoding each constraint independently. To the best of our knowledge, this is the first SAT encoding method for the sequence of AMK cardinality constraints with a ladder shape where $k > 1$.
- Additionally, we present solutions to the Anti-bandwidth Problem and Nurse Rostering Problem, using our new SAT encoding for the sequence of At-Most-One and At-Most-K cardinality constraints, respectively. Experimental results show that our approach outperforms existing SAT encodings, as well as Constraint Programming (CP) and Mixed Integer Programming (MIP) methods that utilize powerful solvers such as CPLEX, CP-SAT, and Gurobi.

This paper extends our previously published research [16] in the following ways: First, we generalize the encoding from AMO constraints to AMK constraints with $k \geq 1$ for sequences of cardinality constraints in a ladder shape, whereas the previous work focused solely on AMO constraints. Second, we also include an experiment on the Nurse Rostering Problem to evaluate the performance of the proposed encoding on combinatorial problems that include sequences of AMK constraints. Third, for the Anti-bandwidth Problem, we develop a parallel search implementation, leading to improved performance. We also expanded our experiment to include comparisons and evaluations with *Adder Network* encoding [17] and powerful CP and MIP solvers, namely *Gurobi* and *CP-SAT*. This comparison highlights the effectiveness of our proposed SAT encoding in solving the Anti-bandwidth Problem and Nurse Rostering Problem when compared to CP and MIP solvers.

The paper is organized as follows: Section 1 introduces the research problem and summarizes the contributions of this study. Section 2 presents the basic definitions of the Ladder AMK constraint and discusses how to represent one AMK constraint using two sub-AMK constraints. Section 3 details the block encoding for the Ladder AMK constraint and includes a comparative evaluation against existing SAT encodings. Section 4 introduces the encoding for the Ladder AMO constraint, which is a specific case of the Ladder AMK constraint with $k = 1$. Sections 5 and 6 sequentially explain the approach to solving the Anti-bandwidth Problem and Nurse Rostering Problem

using SAT solving. Section 7 provides experimental results that compare our proposed encoding with existing SAT encodings, as well as state-of-the-art CP and MIP solvers. Finally, Section 8 concludes the paper.

2 Preliminaries

2.1 Ladder AMK constraint

A Ladder AMK constraint is defined as a sequence of AMK cardinality constraints that involve w boolean variables. In this structure, each pair of adjacent AMK constraints shares $w - 1$ variables. They differ only in the first variable of the first constraint and the last variable of the second constraint. This overlapping feature results in a ladder-like shape. The formal definition of the Ladder AMK constraint is provided in Definition 1.

Definition 1. *Given a sequence of n boolean variables $\Omega = \{x_1, x_2, \dots, x_n\}$ and a number w such that $1 < w \leq n$, a Ladder AMK constraint of width w is formulated as follows:*

$$\text{Ladder}(\Omega, w) = \bigwedge_{i=0}^{n-w} \left(\sum_{j=i+1}^{i+w} x_j \leq k \right) \quad 1 \leq k < w$$

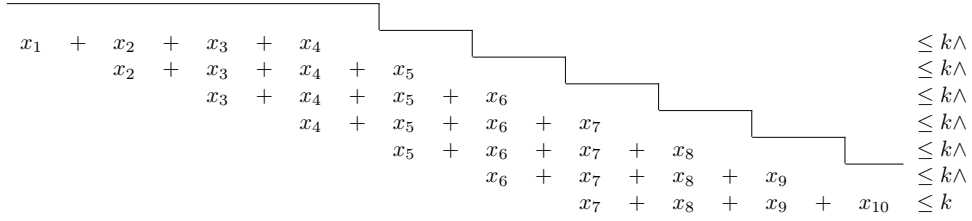


Fig. 1: Ladder AMK constraint of 10 variables and width 4.

Figure 1 illustrates a Ladder AMK constraint over a sequence of 10 boolean variables $\Omega = \{x_1, x_2, \dots, x_{10}\}$ of width $w = 4$. This Ladder AMK constraint consists of seven AMK constraints, each involving four consecutive variables in the sequence.

As discussed in Section 1, a wide range of combinatorial problems can be modeled using sequence constraints, particularly the Ladder AMK constraint. For instance, in the Car Sequencing Problem [1, 2], let $c_{i,o}$ be a Boolean variable that indicates whether the i^{th} car is equipped with option o , i.e., $c_{i,o}$ is *true* if the i^{th} car is equipped with option o and *false* otherwise. Given a production sequence of 10 cars, the constraint requiring that, in any 5 consecutive cars, at most 2 cars are equipped with the option o can be modeled as a Ladder AMK constraint with $n = 10$, $w = 5$, and $k = 2$, as illustrated in Figure 2.

In the Nurse Rostering Problem [3–5], let $x_{i,j,s}$ be a Boolean variable indicating whether nurse i is assigned to shift $s \in \{D, E, N, O\}$ on day j^{th} , where D , E , and

$$\begin{array}{rcl}
c_{1,o} + c_{2,o} + c_{3,o} + c_{4,o} + c_{5,o} & \leq & 2 \wedge \\
c_{2,o} + c_{3,o} + c_{4,o} + c_{5,o} + c_{6,o} & \leq & 2 \wedge \\
c_{3,o} + c_{4,o} + c_{5,o} + c_{6,o} + c_{7,o} & \leq & 2 \wedge \\
c_{4,o} + c_{5,o} + c_{6,o} + c_{7,o} + c_{8,o} & \leq & 2 \wedge \\
c_{5,o} + c_{6,o} + c_{7,o} + c_{8,o} + c_{9,o} & \leq & 2 \wedge \\
c_{6,o} + c_{7,o} + c_{8,o} + c_{9,o} + c_{10,o} & \leq & 2
\end{array}$$

Fig. 2: Ladder AMK constraint ensuring that for any 5 consecutive cars in a sequence of 10 cars, at most 2 cars can be equipped with option o .

N denote day, evening, and night shifts, respectively, while O represents an off-day. Given a planning horizon of 28 consecutive days, the constraint requiring that nurse i is assigned to at most 8 night shifts in any 14 consecutive days can be modeled as a Ladder AMK constraint with $n = 28$, $w = 14$, and $k = 8$, as illustrated in Figure 3.

$$\begin{array}{rcl}
x_{i,1,N} + x_{i,2,N} + \dots + x_{i,14,N} & \leq & 8 \wedge \\
x_{i,2,N} + \dots + x_{i,14,N} + x_{i,15,N} & \leq & 8 \wedge \\
\dots & \leq & 8 \wedge \\
x_{i,14,N} + x_{i,15,N} + \dots + x_{i,27,N} & \leq & 8 \wedge \\
x_{i,15,N} + \dots + x_{i,27,N} + x_{i,28,N} & \leq & 8
\end{array}$$

Fig. 3: Ladder AMK constraint ensuring that nurse i works at most 8 night shifts in any 14 consecutive days over a 28-day planning horizon.

In addition to the examples above, many constraints arising in other combinatorial and scheduling problems can also be modeled as Ladder AMK constraints, such as the Crew Rostering Problem [6], the taxi driver shift scheduling problem [7], the RobinX sports timetabling problem [8], and the Anti-bandwidth Problem [14, 16], as discussed in Section 1. In general, any requirement that limits the number of selected activities, resources, or events within a window over an ordered sequence can be expressed in this form. Therefore, the Ladder AMK constraint is not restricted to specific benchmark problems, but constitutes a general and practically relevant modeling primitive. Its ability to capture sequence cardinality restrictions highlights its wide applicability and practical significance in real-world optimization problems.

2.2 Decomposition of AMK constraint

An AMK constraint can be decomposed into two separate AMK constraints applied to two subsets of the original sequence of variables. This decomposition is formally explained in Proposition 1. It also incorporates connecting clauses to ensure that the decomposed constraints comply with the original constraint without violating it.

Proposition 1. *For any integer number i s.t. $1 \leq i < w$, the constraint*

$$x_1 + x_2 + \dots + x_w \leq k$$

holds if and only if the following constraint holds:

$$(x_1 + \dots + x_i \leq k) \wedge (x_{i+1} + \dots + x_w \leq k)$$

$$\wedge \bigwedge_{j=1}^k (x_1 + \dots + x_i \leq k - j \vee x_{i+1} + \dots + x_w \leq j - 1)$$

Proposition 1 can be proven to be correct by considering all possible cases in which one of the two sub-expressions has a total of $k, k - 1, k - 2, \dots, 1$. In each of these cases, the other sub-expression must have a corresponding total that is less than or equal to $0, 1, 2, \dots, k - 1$, thereby the sum of these two expressions never exceeds k . The following example illustrates the decomposition of an AMK constraint with $k = 3$ over 8 variables $\{x_1, \dots, x_8\}$.

$$\sum_{i=1}^8 x_i \leq 3 \quad \equiv \quad \begin{aligned} & (x_1 + x_2 + x_3 + x_4 \leq 3) \wedge (x_5 + x_6 + x_7 + x_8 \leq 3) \\ & \wedge ((x_1 + x_2 + x_3 + x_4 \leq 2) \vee (x_5 + x_6 + x_7 + x_8 \leq 0)) \\ & \wedge ((x_1 + x_2 + x_3 + x_4 \leq 1) \vee (x_5 + x_6 + x_7 + x_8 \leq 1)) \\ & \wedge ((x_1 + x_2 + x_3 + x_4 \leq 0) \vee (x_5 + x_6 + x_7 + x_8 \leq 2)), \end{aligned}$$

The decomposition of the AMK constraint serves a crucial role in the proposed encoding. In subsequent sections, the proposed encoding leverages this decomposition to replace a large AMK constraint with smaller sub-constraints that can be reused across consecutive constraints. As a result, the proposed encoding achieves a reduced encoding size of the resulting SAT formulation.

The Ladder AMO constraint is a specific case of the Ladder AMK constraint, which occurs when $k = 1$. To derive the decomposition of the AMO constraint, one can simply substitute k with 1 in the decomposition of the AMK constraint. Lemma 1 provides a formal decomposition of an AMO constraint that involves w variables.

Lemma 1. *For any integer number i s.t. $1 \leq i < w$, the constraint*

$$x_1 + x_2 + \dots + x_w \leq 1$$

holds if and only if the following constraint holds:

$$(x_1 + \dots + x_i \leq 1) \wedge (x_{i+1} + \dots + x_w \leq 1)$$

$$\wedge (x_1 + \dots + x_i \leq 0 \vee x_{i+1} + \dots + x_w \leq 0)$$

3 Sequential Counter encoding for Ladder constraint

This section introduces the proposed encoding called **SCL** (*Sequential Counter encoding for Ladder constraints*) for encoding Ladder AMK constraints. The SCL encoding

leverages shared variables among neighboring AMK constraints to partition a Ladder AMK constraint into blocks. These blocks are then represented using an encoding inspired by the *Sequential Counter* [15], which we refer to as the *SC* encoding. After that, auxiliary variables generated by the *SC* encoding are incorporated into clauses that connect adjacent blocks through Proposition 1, thus reformulating the original constraint. By exploiting block decomposition to encode the sequence of AMK constraints as a whole, instead of encoding each AMK constraint independently, the proposed encoding achieves a more compact and efficient representation.

3.1 Partitioning the Ladder AMK constraint into blocks

We define a block consisting of a sequence of w variables $\{v_1, \dots, v_w\}$ as a set of w expressions $\{\mathcal{E}_1, \dots, \mathcal{E}_w\}$ where $\forall i \in [1, w]$, $\mathcal{E}_i = \sum_{j=1}^i v_j$. Figure 4 illustrates the structure of such a block. A block can be efficiently represented by encoding the constraint $\mathcal{E}_w \leq k$ using a single *Sequential Counter* (*Seq*) encoding [15], as *Seq* encoding of $\mathcal{E}_w \leq k$ sequentially encodes $\mathcal{E}_1 \leq k, \dots, \mathcal{E}_{w-1} \leq k$.

$$\begin{array}{rcl} \mathcal{E}_1 & = & \boxed{v_1} \\ \mathcal{E}_2 & = & \boxed{v_1 + v_2} \\ \dots & = & \boxed{\dots} \\ \mathcal{E}_w & = & \boxed{v_1 + v_2 + \dots + v_w} \end{array}$$

Fig. 4: A block consisting of a sequence of w variables $\{v_1, \dots, v_w\}$.

Leverage this insight, we partition the input Ladder AMK constraint into blocks. Given a Ladder AMK constraint with width w over a sequence of n variables $\Omega = \{x_1, x_2, \dots, x_n\}$, we first divide Ω into $M = \lceil \frac{n}{w} \rceil$ subsets $\{\omega_1, \dots, \omega_M\}$. Each subset ω_i ($1 \leq i \leq M - 1$) contains exactly w unique consecutive variables s.t. $\omega_i = \{x_{i,1}, \dots, x_{i,w}\}$, while the last subset ω_M may contain fewer variables if n is not divisible by w . We then consider each subset ω_i as sequences of variables and construct blocks over these sequences so that the resulting blocks collectively cover all expressions required by the Ladder AMK constraint.

For instance, Figure 5 illustrates the partitioning of the Ladder AMK constraint presented in Figure 1. The sequence of variables in this constraint is divided into three subsets $\{\{x_1, \dots, x_4\}, \{x_5, \dots, x_8\}, \{x_9, x_{10}\}\}$. From these subsets, we construct blocks by considering appropriate variable sequences. The first subset (resp. the last subset) is considered as the sequence $\{x_4, \dots, x_1\}$ (resp. $\{x_9, x_{10}\}$), from which we construct block A_1 (resp. block A_4). For the second subset, to cover all expressions required by the Ladder AMK constraint, we consider two sequences, namely $\{x_5, \dots, x_8\}$ and $\{x_8, \dots, x_5\}$, and construct two blocks A_2 and A_3 , respectively. Since each block can be efficiently represented using a single *Seq* encoding, the Ladder AMK constraint illustrated in Figure 5 can be encoded using four *Seq* encodings, instead of the seven encodings that would be required if all seven AMK constraints were encoded independently.

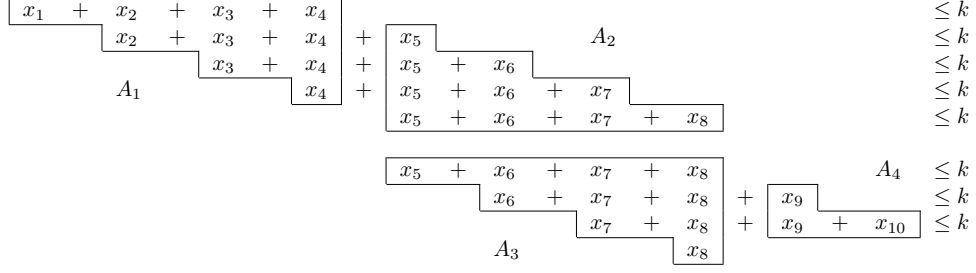


Fig. 5: Decomposition of an Ladder AMK constraint with $n = 10$ and $w = 4$.

3.2 Encoding for Ladder AMK constraint

This section is organized into two parts. First, we describe how each individual block obtained from the decomposition of the Ladder AMK constraint is encoded using *SC* encoding. Second, we connect adjacent blocks to reconstruct the original Ladder AMK constraint.

a. Encoding each block in the Ladder AMK constraint

We denote by $x_{i,j}$ the j^{th} variable in the variable sequence of the block i . For instance, consider the block A_2 illustrated in Figure 5. The variable sequence of this block is $\{x_5, x_6, x_7, x_8\}$, which implies that $x_{2,1} \equiv x_5$, $x_{2,2} \equiv x_6$, $x_{2,3} \equiv x_7$, and $x_{2,4} \equiv x_8$. Similarly, block A_3 in Figure 5 has variable sequence $\{x_8, x_7, x_6, x_5\}$. Consequently, $x_{3,1} \equiv x_8$, $x_{3,2} \equiv x_7$, $x_{3,3} \equiv x_6$, and $x_{3,4} \equiv x_5$.

Since each $x_{i,j}$ is a binary variable, the sum $x_{i,1} + \dots + x_{i,j}$ can take any integer value between 0 and j . To represent this sum, we introduce j register bits, denoted by $R_{i,j,s}$ for $s \in \{1, \dots, j\}$. Each register bit $R_{i,j,s}$ indicates that under the condition “the sum of the first j variables in the variable sequence of block i is greater than or equal to s ”. Particularly, $R_{i,j,s}$ is *true* if and only if $x_{i,1} + \dots + x_{i,j} \geq s$, and *false* otherwise, i.e., when $x_{i,1} + \dots + x_{i,j} \leq s - 1$.

It is necessary to mention that all the expressions within each block are At-Most-K. Consequently, when encoding sub-expression $x_{i,1} + \dots + x_{i,j}$ with $k < j$, it is sufficient to introduce only k register bits $\{R_{i,j,1}, \dots, R_{i,j,k}\}$. Any use of a register bit $R_{i,j,s}$ with $s > k$ is redundant, as such bits are always *false*. For example, to encode $x_{1,1} + x_{1,2} + x_{1,3} \leq 2$, only two register bits, namely $R_{1,3,1}$ and $R_{1,3,2}$, are required.

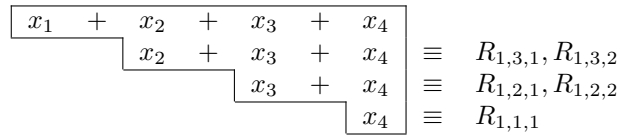


Fig. 6: Register bit construction of block A_1 under the condition ≤ 2 .

Figure 6 illustrates the assignment of register bits for the block A_1 of the Ladder constraint illustrated in Figure 5, under the condition ≤ 2 . x_4 has a maximum value of 1, and thus requires only one register bit, denoted $R_{1,1,1}$. Next, $x_3 + x_4$ has a maximum value of 2, and therefore is assigned two register bits, $R_{1,2,1}$ and $R_{1,2,2}$. For $x_2 + x_3 + x_4$, although the maximum value is 3, due to the constraint ≤ 2 , this sub-expression is also assigned only two register bits, namely $R_{1,3,1}$ and $R_{1,3,2}$. Finally, $x_1 + x_2 + x_3 + x_4$ does not require additional register bits, since condition ≤ 2 can be expressed based on the value of x_1 and the register bits already assigned to $x_2 + x_3 + x_4$.

We also note that, if $R_{i,j,s}$ is *true*, then $R_{i,j,s-1}$ must also be *true*, since $x_{i,1} + \dots + x_{i,j} \geq s$ implies $x_{i,1} + \dots + x_{i,j} \geq s-1$. Besides, the equality $x_{i,1} + \dots + x_{i,j} = s$ is valid if and only if $R_{i,j,s}$ is *true* and $R_{i,j,s+1}$ is *false*. Specifically, $R_{i,j,s}$ is *true* indicating that $x_{i,1} + \dots + x_{i,j} \geq s$, while $R_{i,j,s+1}$ is *false* implying that $x_{i,1} + \dots + x_{i,j} \leq (s+1) - 1 = s$. Combining these two conditions results in $s \leq x_{i,1} + \dots + x_{i,j} \leq s$, and thus $x_{i,1} + \dots + x_{i,j} = s$.

The *SC* encoding for decomposed blocks involves the application of seven equations (1), (2), (3), (4), (5), (6), and (7). The formal proof of the correctness of this encoding is given in Appendix A.

$$\bigwedge_{j=1}^{w-1} x_{i,j} \rightarrow R_{i,j,1} \quad (1)$$

$$\bigwedge_{j=2}^{w-1} \bigwedge_{s=1}^{\min(j-1,k)} R_{i,j-1,s} \rightarrow R_{i,j,s} \quad (2)$$

$$\bigwedge_{j=2}^{w-1} \bigwedge_{s=2}^{\min(j,k)} x_{i,j} \wedge R_{i,j-1,s-1} \rightarrow R_{i,j,s} \quad (3)$$

$$\bigwedge_{j=1}^k \neg x_{i,j} \rightarrow \neg R_{i,j,j} \quad (4)$$

$$\bigwedge_{j=2}^{w-1} \bigwedge_{s=2}^{\min(j,k)} \neg R_{i,j-1,s-1} \rightarrow \neg R_{i,j,s} \quad (5)$$

$$\bigwedge_{j=2}^{w-1} \bigwedge_{s=1}^{\min(j-1,k)} \neg x_{i,j} \wedge \neg R_{i,j-1,s} \rightarrow \neg R_{i,j,s} \quad (6)$$

$$\bigwedge_{j=k+1}^w x_{i,j} \rightarrow \neg R_{i,j-1,k} \quad (7)$$

Equation (1) specifies that the sum of the first j variables in block i must be at least one if $x_{i,j}$ is *true*. Equation (2) states that if the sum of the first $j-1$ variables in block i is greater than or equal to s , then the sum of the first j variables must also be at least s . Equation (3) ensures that if $x_{i,j}$ is *true* and the sum of the first $j-1$ variables is greater than or equal to $s-1$, then the sum

of the first j variables in block i must be at least s . Together, equations (1), (2), and (3) ensure the correctness of the statement “ $R_{i,j,s}$ holds *true* if and only if $x_{i,1} + \dots + x_{i,j} \geq s$ ”.

Next, equation (4) limits the sum of the first j variables in block i to at most $j - 1$ if $x_{i,j}$ is *false*. The equation (5) states that if the sum of the first $j - 1$ variables is less than $s - 1$, then the sum of the first j variables must be less than s . Equation (6) states that if $x_{i,j}$ is *false* and the sum of the first $j - 1$ variables is less than s , then the sum of the first j variables must also be less than s . Taken together, the equations (4), (5), and (6) validate the correctness of the statement “ $R_{i,j,s}$ is *false* if and only if $x_{i,1} + \dots + x_{i,j} \leq s - 1$ ”.

Finally, the equation (7) stipulates that if $x_{i,j}$ is *true*, then the sum of the first $j - 1$ variables in block i must be strictly less than k . By applying this constraint to all indices j within the range of $[k + 1, w]$, we ensure that no more than k out of the w variables can be set to *true*. Additionally, the sizes of the clauses (i.e., the number of literals in each clause) in our encoding are at most 3.

As previously mentioned, the *SC* encoding is inspired by the *Seq* encoding [15]. In particular, equations (1), (2), (3), and (7) are adopted directly from the *Seq* encoding [15]. The remaining equations (4), (5), and (6) are intentionally included to explicitly characterize the conditions under which the corresponding register bits are forced to *false*. This is crucial for ensuring the correctness of the proposed encoding, as these register bits will be reused to represent the block connections.

It is necessary to note that when two blocks involve the same subset of variables $\omega_i = \{x_{i,1}, \dots, x_{i,w}\}$, if one block has already enforced the constraint $x_{i,1} + \dots + x_{i,w} \leq k$ via equation (7), the application of equation (7) in the other block is unnecessary. Exploiting this insight, when encoding two different blocks that share the same subset of variables, we include equation (7) on only one of them. We refer to the block on which equation (7) is applied to enforce the AMK constraint as the AMK block, while the other block, which primarily provides expressions for block connection, is referred to as the connection block.

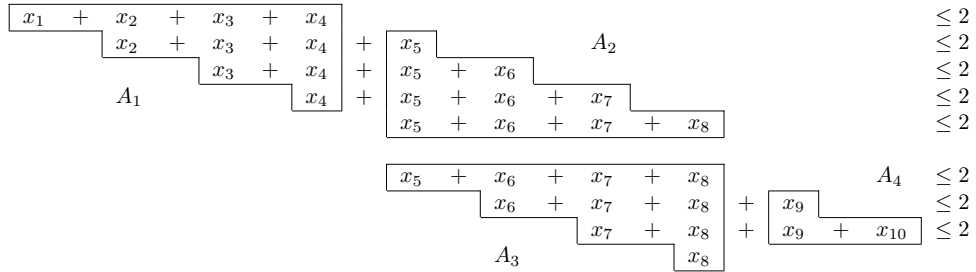


Fig. 7: Decomposition of Ladder At-Most-Two constraint.

Figure 7 illustrates an example of a Ladder AMK constraint with $k = 2$. This Ladder AMK constraint is divided into four blocks, labeled A_1 , A_2 , A_3 , and A_4 . Figure 8 presents these four blocks along with the necessary register bits used in the *SC* encoding. The block A_1 is encoded using *SC* with the formulas from

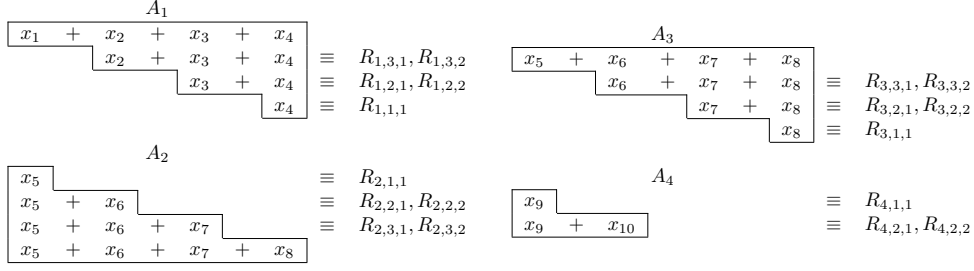


Fig. 8: Register bits constructing of Ladder At-Most-Two constraint.

equations (1) to (7). The generated clauses for constructing the block A_1 are presented below. In this context, the variables are defined as $x_{1,1} \equiv x_4$, $x_{1,2} \equiv x_3$, $x_{1,3} \equiv x_2$, $x_{1,4} \equiv x_1$, $x_{1,1} \equiv R_{1,1,1}$, and $x_4 \equiv R_{1,1,1}$.

$$\begin{aligned}
(1) \quad & \xleftrightarrow{A_1} \quad x_{1,2} \rightarrow R_{1,2,1} \iff x_3 \rightarrow R_{1,2,1} \\
& \quad \quad \quad x_{1,3} \rightarrow R_{1,3,1} \iff x_2 \rightarrow R_{1,3,1} \\
(2) \quad & \xleftrightarrow{A_1} \quad R_{1,1,1} \rightarrow R_{1,2,1} \iff x_4 \rightarrow R_{1,2,1} \\
& \quad \quad \quad R_{1,2,1} \rightarrow R_{1,3,1} \iff R_{1,2,1} \rightarrow R_{1,3,1} \\
& \quad \quad \quad R_{1,2,2} \rightarrow R_{1,3,2} \iff R_{1,2,2} \rightarrow R_{1,3,2} \\
(3) \quad & \xleftrightarrow{A_1} \quad x_{1,2} \wedge R_{1,1,1} \rightarrow R_{1,2,2} \iff x_3 \wedge x_4 \rightarrow R_{1,2,2} \\
& \quad \quad \quad x_{1,3} \wedge R_{1,2,1} \rightarrow R_{1,3,2} \iff x_2 \wedge R_{1,2,1} \rightarrow R_{1,3,2} \\
(4) \quad & \xleftrightarrow{A_1} \quad \neg x_{1,2} \rightarrow \neg R_{1,2,2} \iff \neg x_3 \rightarrow \neg R_{1,2,2} \\
(5) \quad & \xleftrightarrow{A_1} \quad \neg R_{1,1,1} \rightarrow \neg R_{1,2,2} \iff \neg x_4 \rightarrow \neg R_{1,2,2} \\
& \quad \quad \quad \neg R_{1,2,1} \rightarrow \neg R_{1,3,2} \iff \neg R_{1,2,1} \rightarrow \neg R_{1,3,2} \\
(6) \quad & \xleftrightarrow{A_1} \quad \neg x_{1,2} \wedge \neg R_{1,1,1} \rightarrow \neg R_{1,2,1} \iff \neg x_3 \wedge \neg x_4 \rightarrow \neg R_{1,2,1} \\
& \quad \quad \quad \neg x_{1,3} \wedge \neg R_{1,2,1} \rightarrow \neg R_{1,3,1} \iff \neg x_2 \wedge \neg R_{1,2,1} \rightarrow \neg R_{1,3,1} \\
& \quad \quad \quad \neg x_{1,3} \wedge \neg R_{1,2,2} \rightarrow \neg R_{1,3,2} \iff \neg x_2 \wedge \neg R_{1,2,2} \rightarrow \neg R_{1,3,2} \\
(7) \quad & \xleftrightarrow{A_1} \quad x_{1,3} \rightarrow \neg R_{1,2,2} \iff x_2 \rightarrow \neg R_{1,2,2} \\
& \quad \quad \quad x_{1,4} \rightarrow \neg R_{1,3,2} \iff x_1 \rightarrow \neg R_{1,3,2}
\end{aligned}$$

Encoding the block A_2 is the same as A_1 , including the following clauses:

$$\begin{aligned}
(1) \quad & \xleftrightarrow{A_2} \quad x_{2,2} \rightarrow R_{2,2,1} \iff x_6 \rightarrow R_{2,2,1} \\
& \quad \quad \quad x_{2,3} \rightarrow R_{2,3,1} \iff x_7 \rightarrow R_{2,3,1} \\
(2) \quad & \xleftrightarrow{A_2} \quad R_{2,1,1} \rightarrow R_{2,2,1} \iff x_5 \rightarrow R_{2,2,1} \\
& \quad \quad \quad R_{2,2,1} \rightarrow R_{2,3,1} \iff R_{2,2,1} \rightarrow R_{2,3,1} \\
& \quad \quad \quad R_{2,2,2} \rightarrow R_{2,3,2} \iff R_{2,2,2} \rightarrow R_{2,3,2} \\
(3) \quad & \xleftrightarrow{A_2} \quad x_{2,2} \wedge R_{2,1,1} \rightarrow R_{2,2,2} \iff x_6 \wedge x_5 \rightarrow R_{2,2,2} \\
& \quad \quad \quad x_{2,3} \wedge R_{2,2,1} \rightarrow R_{2,3,2} \iff x_7 \wedge R_{2,2,1} \rightarrow R_{2,3,2} \\
(4) \quad & \xleftrightarrow{A_2} \quad \neg x_{2,2} \rightarrow \neg R_{2,2,2} \iff \neg x_6 \rightarrow \neg R_{2,2,2}
\end{aligned}$$

$$\begin{aligned}
(5) \quad & \begin{array}{ccc} \xleftrightarrow{A_2} & \neg R_{2,1,1} & \rightarrow \neg R_{2,2,2} \\ & \neg R_{2,2,1} & \rightarrow \neg R_{2,3,2} \end{array} \iff \begin{array}{ccc} \neg x_5 & \rightarrow & \neg R_{2,2,2} \\ \neg R_{2,2,1} & \rightarrow & \neg R_{2,3,2} \end{array} \\
(6) \quad & \begin{array}{ccc} \xleftrightarrow{A_2} & \neg x_{2,2} \wedge \neg R_{2,1,1} & \rightarrow \neg R_{2,2,1} \\ & \neg x_{2,3} \wedge \neg R_{2,2,1} & \rightarrow \neg R_{2,3,1} \\ & \neg x_{2,3} \wedge \neg R_{2,2,2} & \rightarrow \neg R_{2,3,2} \end{array} \iff \begin{array}{ccc} \neg x_6 \wedge \neg x_5 & \rightarrow & \neg R_{2,2,1} \\ \neg x_7 \wedge \neg R_{2,2,1} & \rightarrow & \neg R_{2,3,1} \\ \neg x_7 \wedge \neg R_{2,2,2} & \rightarrow & \neg R_{2,3,2} \end{array} \\
(7) \quad & \begin{array}{ccc} \xleftrightarrow{A_2} & x_{2,3} & \rightarrow \neg R_{2,2,2} \\ & x_{2,4} & \rightarrow \neg R_{2,3,2} \end{array} \iff \begin{array}{ccc} x_7 & \rightarrow & \neg R_{2,2,2} \\ x_8 & \rightarrow & \neg R_{2,3,2} \end{array}
\end{aligned}$$

Since the block A_2 has already enforced the constraint $x_5 + x_6 + x_7 + x_8 \leq 2$, the block A_3 only leverages the formulas from equations (1) to (6) to generate the necessary register bits for its connection to the adjacent block A_4 . Finally, the block A_4 is a special case, as it contains only two variables instead of four as in the other blocks. Consequently, it is encoded using seven equations (1) to (7), with w set to 2. As a result, the encoding of block A_4 consists of the following clauses:

$$\begin{aligned}
& (x_9 \rightarrow R_{4,2,1}) \wedge (x_{10} \rightarrow R_{4,2,1}) \wedge ((x_9 \wedge x_{10}) \rightarrow R_{4,2,2}) \wedge (\neg x_9 \rightarrow \neg R_{4,2,2}) \\
& \wedge (\neg x_{10} \rightarrow \neg R_{4,2,2}) \wedge ((\neg x_9 \wedge \neg x_{10}) \rightarrow \neg R_{4,2,1}).
\end{aligned}$$

b. *Clauses connecting two adjacent blocks*

After encoding blocks, we construct connecting clauses to connect adjacent blocks, thereby reformulate the Ladder AMK constraint. This construction is supported by Proposition 1. For instance, connection between blocks A_1 and A_2 in Figure 7 is formed as follows.

$$\begin{aligned}
(x_2 + x_3 + x_4 + x_5 \leq 2) & \stackrel{\text{Prop. 1}}{\equiv} (x_2 + x_3 + x_4 \leq 2) \wedge (x_5 \leq 2) \\
& \wedge (x_2 + x_3 + x_4 \leq 1 \vee x_5 \leq 0) \\
& \wedge (x_2 + x_3 + x_4 \leq 0 \vee x_5 \leq 1) \\
(x_3 + x_4 + x_5 + x_6 \leq 2) & \stackrel{\text{Prop. 1}}{\equiv} (x_3 + x_4 \leq 2) \wedge (x_5 + x_6 \leq 2) \\
& \wedge (x_3 + x_4 \leq 1 \vee x_5 + x_6 \leq 0) \\
& \wedge (x_3 + x_4 \leq 0 \vee x_5 + x_6 \leq 1) \\
(x_4 + x_5 + x_6 + x_7 \leq 2) & \stackrel{\text{Prop. 1}}{\equiv} (x_4 \leq 2) \wedge (x_5 + x_6 + x_7 \leq 2) \\
& \wedge (x_4 \leq 1 \vee x_5 + x_6 + x_7 \leq 0) \\
& \wedge (x_4 \leq 0 \vee x_5 + x_6 + x_7 \leq 1)
\end{aligned}$$

Since the constraint $x_1 + x_2 + x_3 + x_4 \leq 2$ (resp. $x_5 + x_6 + x_7 + x_8 \leq 2$) is *true*, as guaranteed by the equation (7), it follows that all its sub-constraints are also *true*, namely $x_2 + x_3 + x_4 \leq 2$, $x_3 + x_4 \leq 2$, and $x_4 \leq 2$ (resp. $x_5 + x_6 + x_7 \leq 2$, $x_5 + x_6 \leq 2$, and $x_5 \leq 2$). Consequently, the connection between block A_1 and block A_2 can be represented as follows.

$$\begin{aligned}
& (x_2 + x_3 + x_4 \leq 1 \vee x_5 \leq 0) \wedge (x_2 + x_3 + x_4 \leq 0 \vee x_5 \leq 1) \\
& \wedge (x_3 + x_4 \leq 1 \vee x_5 + x_6 \leq 0) \wedge (x_3 + x_4 \leq 0 \vee x_5 + x_6 \leq 1) \\
& \wedge (x_4 \leq 1 \vee x_5 + x_6 + x_7 \leq 0) \wedge (x_4 \leq 0 \vee x_5 + x_6 + x_7 \leq 1)
\end{aligned}$$

We utilize the generated register bits to express the connection between A_1 and A_2 . For instance, the expression $x_2 + x_3 + x_4 \leq 1$ serves as the negation of $x_2 + x_3 + x_4 \geq 2$ and can be replaced with $\neg R_{1,3,2}$. Similarly, the expression $x_2 + x_3 + x_4 \leq 0$ is the negation of $x_2 + x_3 + x_4 \geq 1$ and can be represented as $\neg R_{1,3,1}$. This substitution can be applied to the remaining expressions as well. Therefore, the connection between block A_1 and block A_2 can be expressed more straightforwardly as follows.

$$\begin{aligned} & (\neg R_{1,3,2} \vee x_5 \leq 0) \wedge (\neg R_{1,3,1} \vee x_5 \leq 1) \\ & \wedge (\neg R_{1,2,2} \vee \neg R_{2,2,1}) \wedge (\neg R_{1,2,1} \vee \neg R_{2,2,2}) \\ & \wedge (x_4 \leq 1 \vee \neg R_{2,3,1}) \wedge (x_4 \leq 0 \vee \neg R_{2,3,2}) \end{aligned}$$

In summary, connecting two consecutive blocks i and $i+1$ in the Ladder AMK constraint involves a series of disjunctions, as formally presented in equation (8). This equation can also be optimized by reusing register bits from the SC encoding of the individual blocks, as presented in the example above.

$$\bigwedge_{j=2}^w \bigwedge_{p=1}^k \left(\sum_{q=1}^{w-j+1} x_{i,q} \leq k-p \vee \sum_{q'=1}^{j-1} x_{i+1,q'} \leq p-1 \right) \quad (8)$$

3.3 Comparison of Ladder AMK constraint encodings

In this section, we assess the performance of the SCL encoding on the Ladder AMK constraint ($SCL_{\leq k}$) and compare it with other encodings that encode each AMK constraint individually. These encodings include *Pairwise*, *Sequential Counter* (*Seq*) [15], *Binary Decision Diagrams* (*BDD*) [18], *Cardinality Network* (*Card*) [19, 20], and *Adder Network* (*Adder*) [17].

Table 1 summarizes the evaluation of different encodings used to represent the Ladder AMK constraints over n variables with width w . The encodings are compared based on the number of required auxiliary variables, the number of generated clauses, and the complexity of the clause space in the worst case, specifically when w is approximately $\frac{n}{2}$.

Pairwise encoding generates a clause for every set of $k+1$ variables among the w variables in each AMK constraint. These clauses ensure that at least one variable in any set of $k+1$ variables is assigned a value of false. This means that no more than k variables can be assigned the value true. By eliminating redundant constraints, *Pairwise* encoding uses C_{k+1}^w clauses to encode the first AMK constraint and C_k^{w-1} clauses to encode each of the remaining $N-1$ AMK constraints. As a result, the total number of clauses is $C_{k+1}^w + (N-1)C_k^{w-1}$, leading to a complexity of $\mathcal{O}(n^{k+1})$.

Sequential Counter (*Seq*) encoding [15] employs a sequential counting technique. In the context of the AMK constraint, this encoding ensures that in any sequence of w consecutive variables, no more than k variables can be assigned *true*, thereby maintaining the AMK property. Each *Seq* encoding for an AMK constraint over w variables requires at most $k(w-2)$ additional variables and generates approximately

Table 1: Size of SAT encoding for a Ladder AMK constraint over n variables with width w , $N = (n - w) + 1$ and $M = \lceil \frac{n}{w} \rceil$.

Encoding	Technique	Auxiliary variables	Clauses	Complexity
<i>Pairwise</i>	Encode each AMK constraint separately	0	$C_{k+1}^w + (N - 1)C_k^{w-1}$	$\mathcal{O}(n^{k+1})$
<i>Seq</i>		$Nk(w - 2)$	$N(3kw - 7k + w)$	$\mathcal{O}(n^2k)$
<i>BDD</i>		$N(w - k)(k + 1)$	$2N(w - k)(k + 1)$	$\mathcal{O}(n^2k)$
<i>Card</i>		$\mathcal{O}(n^2 \log_2^2 k)$	$\mathcal{O}(n^2 \log_2^2 k)$	$\mathcal{O}(n^2 \log_2^2 k)$
<i>Adder</i>		$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$
<i>SCL_{≤k}</i>	Encode set of AMK constraints	$(2M - 2)(wk - \frac{k^2 + k}{2} - 1)$	$9Mkw - 5Mk^2 - Mw - 7Mk - 9kw + 5k^2 - 2M + 2w + 6k + 2$	$\mathcal{O}(nk)$

$3kw - 7k + w$ clauses, leading to a total of $Nk(w - 2)$ auxiliary variables, $N(3kw - 7k + w)$ clauses, and a complexity of $\mathcal{O}(n^2k)$.

Binary Decision Diagrams (BDD) encoding [18] represents the AMK constraint as a circuit of *if-then-else (ITE)* gates. The goal of this encoding is to constrain the circuit's inputs to ensure that signal propagation through the gates ultimately reaches the final gate. For an AMK constraint on w variables, *BDD* encoding constructs a circuit with approximately $(w - k)(k + 1)$ gates, corresponding to the same number of auxiliary variables. Each gate in the circuit has two outputs, representing the boolean values *true* and *false*, leading to a total of approximately $2(w - k)(k + 1)$ clauses. In summary, encoding a Ladder AMK constraint using the *BDD* encoding requires approximately $N(w - k)(k + 1)$ auxiliary variables, $2N(w - k)(k + 1)$ clauses, and has a complexity of $\mathcal{O}(n^2k)$.

Leveraging the odd-even merging technique in *Sorting Network* encoding [21], Asín et al. introduced the *Half Sorting Network* and *Half Merging Network* encodings [20], which reduce the number of clauses by half. Building on these approaches, Asín et al. further proposed the *Simplified Half-Merging Network*, a streamlined version of the *Half Merging Network*. This encoding later served as the foundation for the development of *Cardinality Network (Card)* encoding [20]. For an AMK constraint of w variables, *Card* encoding uses $\mathcal{O}(n \log_2^2 k)$ auxiliary variables and $\mathcal{O}(n \log_2^2 k)$ clauses. When applied to Ladder AMK constraints, this encoding requires a total of $\mathcal{O}(n^2 \log_2^2 k)$ auxiliary variables, $\mathcal{O}(n^2 \log_2^2 k)$ clauses, and has a complexity of $\mathcal{O}(n^2 \log_2^2 k)$.

Adder Network (Adder) encoding [17] translates each AMK constraint into a boolean circuit composed of half-adders and full-adders arranged as a binary adder tree. Through this circuit, a binary representation of the sum of input variables is produced, and a binary comparator circuit is used to ensure that this sum does not exceed k . For an AMK constraint over w variables, *Adder* encoding introduces a $\mathcal{O}(n)$ sized translation [17]. Consequently, encoding a Ladder AMK constraint using *Adder* requires $\mathcal{O}(n^2)$ auxiliary variables and clauses.

Our analysis reveals that the *SCL_{≤k}* encoding results in fewer clauses than other encodings. For simplicity, we assume that the size of the last subset is equal to that of the others, although it is typically smaller. Through block decomposition, one AMK

block is generated for the first subset and one for the last subset. For the remaining $(M - 2)$ subsets, a total of $(M - 2)$ AMK blocks and $(M - 2)$ connection blocks are generated. Each AMK block, which is encoded using equations (1) to (7), requires $4kw - 2k^2 - 4k - 1$ clauses, whereas each connection block, which is encoded using equations (1) to (6), uses $(w - k)$ fewer clauses than the AMK blocks. Furthermore, connecting these $2 + (M - 2) + (M - 2) = 2M - 2$ blocks necessitates $M - 1$ connections, with each connection requiring $(w - k)k$ clauses. In total, the number of clauses employed by the $SCL_{\leq k}$ encoding is given by:

AMK-blocks-clauses	=	$2 + (M - 2)$	x	$(4kw - 2k^2 - 4k - 1)$
Connection-blocks-clauses	=	$(M - 2)$	x	$(4kw - 2k^2 - 4k - 1 - w + k)$
Connecting-clauses	=	$(M - 1)$	x	$(w - k)k$
Total-clauses	=	$9Mkw - 5Mk^2 - Mw - 7Mk - 9kw + 5k^2 - 2M + 2w + 6k + 2$		

In the calculation of auxiliary variable usage, $SCL_{\leq k}$ encoding employs $wk - \frac{k^2+k}{2}$ register bits per block, including both AMK blocks and connection blocks. By referring to Figure 8, we also observe that the first register bit is equivalent to the first variable, i.e., $R_{1,1,1} \equiv x_4$. Leveraging this insight, we can substitute that register bit with its corresponding variable to further decrease the number of auxiliary variables required by each block by one, resulting in $wk - \frac{k^2+k}{2} - 1$. Consequently, the total number of auxiliary variables needed for the entire Ladder constraint becomes $(2M - 2)(wk - \frac{k^2+k}{2} - 1)$.

The analysis above demonstrates the efficiency of our proposed encoding. In terms of the required auxiliary variables and the number of generated clauses, the $SCL_{\leq k}$ encoding outperforms the *Seq*, *BDD*, *Card*, and *Adder* encodings. Specifically, while these four encodings require at least $\mathcal{O}(n^2)$ auxiliary variables and clause space, $SCL_{\leq k}$ only needs $\mathcal{O}(Mwk)$, which is approximately $\mathcal{O}(nk)$. Additionally, although the *Pairwise* encoding does not utilize auxiliary variables, its complexity of $\mathcal{O}(n^{k+1})$ is significantly higher than our linear complexity of $\mathcal{O}(nk)$. We provide a more comprehensive demonstration in the experimental evaluation (Section 7.1.2), which clarifies the comparative analysis and highlights the superiority of $SCL_{\leq k}$ over *Pairwise*, *Seq*, *BDD*, *Card* and *Adder* encodings.

4 Encodings for Ladder AMO constraint

In this section, we introduce the SCL encoding formulas for the Ladder AMO constraint ($SCL_{\leq 1}$), derived from those for the Ladder AMK constraint ($SCL_{\leq k}$). Additionally, we present *Duplex* encoding [14], a decomposition-based encoding that has proven to be an effective encoding for the Ladder AMO constraint.

4.1 $SCL_{\leq 1}$ encoding

a. Encoding each block in the Ladder AMO constraint

The Ladder AMO constraint is essentially a specific instance of the Ladder AMK constraint. Therefore, its block encoding can be directly derived from that of the Ladder AMK. When $k = 1$, the formulas from (9) to (13) below are utilized to encode the i^{th} block in the Ladder AMO constraint.

$$\bigwedge_{j=1}^{w-1} x_{i,j} \rightarrow R_{i,j,1} \stackrel{k=1}{\Rightarrow} \bigwedge_{j=1}^{w-1} x_{i,j} \rightarrow R_{i,j,1} \quad (9)$$

$$\bigwedge_{j=2}^{w-1} \bigwedge_{s=1}^{\min(j-1,k)} R_{i,j-1,s} \rightarrow R_{i,j,s} \stackrel{k=1}{\Rightarrow} \bigwedge_{j=2}^{w-1} R_{i,j-1,1} \rightarrow R_{i,j,1} \quad (10)$$

$$\bigwedge_{j=2}^{w-1} \bigwedge_{s=2}^{\min(j,k)} x_{i,j} \wedge R_{i,j-1,s-1} \rightarrow R_{i,j,s} \stackrel{k=1}{\Rightarrow} \emptyset$$

$$\bigwedge_{j=1}^k \neg x_{i,j} \rightarrow \neg R_{i,j,j} \stackrel{k=1}{\Rightarrow} \neg x_{i,1} \rightarrow \neg R_{i,1,1} \quad (11)$$

$$\bigwedge_{j=2}^{w-1} \bigwedge_{s=2}^{\min(j,k)} \neg R_{i,j-1,s-1} \rightarrow \neg R_{i,j,s} \stackrel{k=1}{\Rightarrow} \emptyset$$

$$\bigwedge_{j=2}^{w-1} \bigwedge_{s=1}^{\min(j-1,k)} \neg x_{i,j} \wedge \neg R_{i,j-1,s} \rightarrow \neg R_{i,j,s} \stackrel{k=1}{\Rightarrow} \bigwedge_{j=2}^{w-1} \neg x_{i,j} \wedge \neg R_{i,j-1,1} \rightarrow \neg R_{i,j,1} \quad (12)$$

$$\bigwedge_{j=k+1}^w x_{i,j} \rightarrow \neg R_{i,j-1,k} \stackrel{k=1}{\Rightarrow} \bigwedge_{j=2}^w x_{i,j} \rightarrow \neg R_{i,j-1,1} \quad (13)$$

b. *Clauses connecting two adjacent blocks*

Similar to the encoding of individual blocks, the connection between blocks in the Ladder AMO constraint can also be derived from the block connection in the Ladder AMK constraint (refer to equation (8)) by setting $k = 1$. Specifically,

$$\bigwedge_{j=2}^w \bigwedge_{p=1}^k \left(\sum_{q=1}^{w-j+1} x_{i,q} \leq k - p \vee \sum_{q'=1}^{j-1} x_{i+1,q'} \leq p - 1 \right)$$

$$\Downarrow k = 1$$

$$\bigwedge_{j=2}^w \left(\sum_{q=1}^{w-j+1} x_{i,q} \leq 0 \vee \sum_{q'=1}^{j-1} x_{i+1,q'} \leq 0 \right) \quad (14)$$

4.2 Duplex encoding

Fazekas et al. introduced the *Duplex* [14] for encoding Ladder AMO constraints. In this approach, the AMO and At-Most-Zero (AMZ) sub-expressions obtained through the

decomposition of Ladder AMO constraints are encoded by applying *Binary Decision Diagrams* (*BDD*) encoding [22, 23] for corresponding windows of variables (note that the term “window” is equivalent to our term “subset”). Each window is encoded using two components: a forward *BDD* and a backward *BDD*. The forward *BDD* is constructed from the leftmost to the rightmost variable, while the backward *BDD* is constructed in the reverse direction, from right to left.

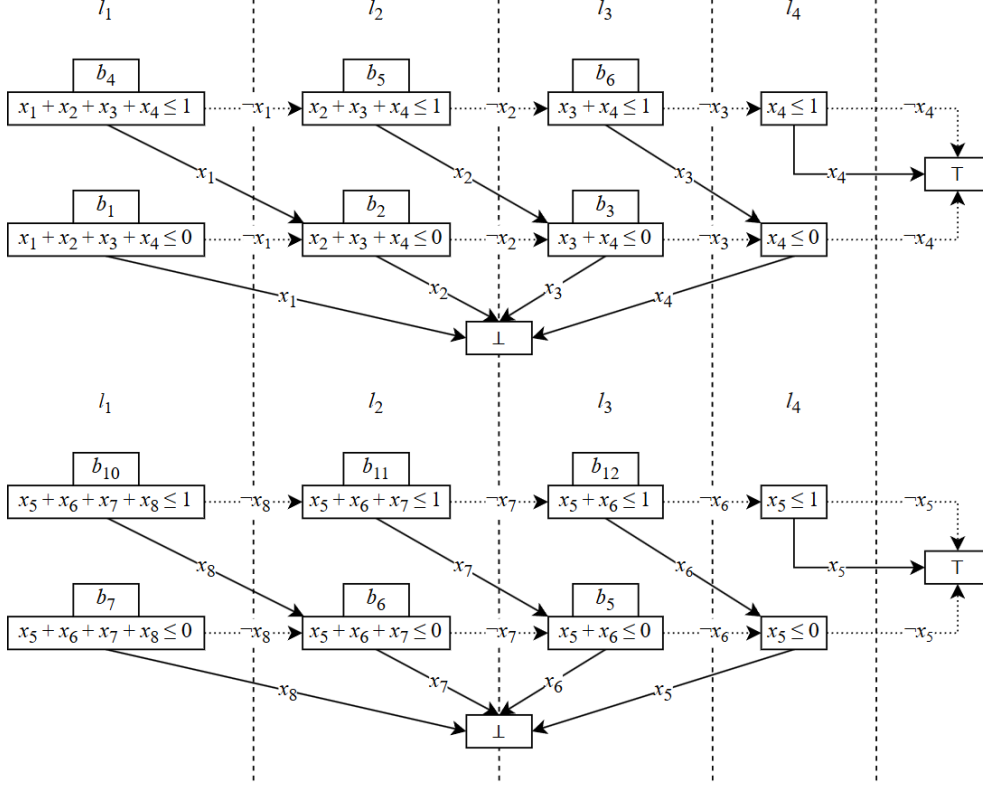


Fig. 9: Forward and backward *BDDs* for Ladder AMO constraint.

Source: Fazekas et al. [14]

Figure 9 illustrates the construction of two *BDDs* with the variable arrangements of $x_1 \prec x_2 \prec x_3 \prec x_4$ and $x_8 \prec x_7 \prec x_6 \prec x_5$, respectively. This construction involves making decisions for each variable, using their values to branch the necessary conditions of the original constraints. For instance, to maintain the condition $x_1 + x_2 + x_3 + x_4 \leq 1$, if x_1 is *true*, then $x_2 + x_3 + x_4 \leq 0$ must hold; conversely, if x_1 is *false*, the condition $x_2 + x_3 + x_4 \leq 1$ must be satisfied.

To ensure there is at most one variable assigned *true* within the AMO constraints of Ladder AMO constraint, *Duplex* encoding bonds the constructed *BDDs* together. Specifically, for each successive pair of windows $\{\omega_i, \omega_{i+1}\}$, where ω_i^f is the forward

BDD of ω_i and ω_{i+1}^b is the backward BDD of ω_{i+1} , the j^{th} layer of ω_i^f is connected to the $(w - j + 2)^{th}$ layer of ω_{i+1}^b , where $2 \leq j \leq w$. For instance, bonding two $BDDs$ as shown in Figure 9 requires the following clauses.

$$\begin{aligned} & \wedge(x_2 + x_3 + x_4 \leq 1) \wedge (x_5 \leq 1) \wedge ((x_2 + x_3 + x_4 \leq 0) \vee (x_5 \leq 0)) \\ & \wedge(x_3 + x_4 \leq 1) \wedge (x_5 + x_6 \leq 1) \wedge ((x_3 + x_4 \leq 0) \vee (x_5 + x_6 \leq 0)) \\ & \wedge(x_4 \leq 1) \wedge (x_5 + x_6 + x_7 \leq 1) \wedge ((x_4 \leq 0) \vee (x_5 + x_6 + x_7 \leq 0)) \end{aligned}$$

Leveraging the benefits of reusing shared sub-expressions, the *Duplex* encoding achieves significantly enhanced complexity compared to non-decompose encodings. As shown in the work of Fazekas et al., the *Duplex* encoding has a complexity of $\mathcal{O}(n)$ [14], whereas all non-decompose encodings exhibit complexity of $\mathcal{O}(n^2)$ (refer to Table 2). For a Ladder AMO constraint involving n variables and considering $M = \lceil \frac{n}{w} \rceil$ windows of width w , with $N = (n - w) + 1$ being the number of AMO constraints, this encoding generates around $13Mw - 14M - 3w + 2$ clauses and introduces additional $4M(w - 1)$ auxiliary variables.

4.3 Encoding comparison for Ladder AMO constraints

In addition to the previously mentioned encodings, which are *Pairwise*, *Sequential Counter (Seq)*, *Binary Decision Diagrams (BDD)*, *Cardinality Network (Card)*, and *Adder Network (Adder)*, we compare $SCL_{\leq 1}$ with the *Duplex* encoding [14]. Moreover, among the group of encodings that individually handle each AMO constraint, we also include the *Product* encoding [24].

Table 2: Size of SAT encoding for a Ladder AMO constraint over n variables with width w , $N = (n - w) + 1$ and $M = \lceil \frac{n}{w} \rceil$.

Encoding	Technique	Auxiliary variables	Clauses	Complexity
<i>Pairwise</i>	Encode each AMO constraint separately	0	$\frac{(w-1)w}{2} + (N-1)(w-1)$	$\mathcal{O}(n^2)$
<i>Seq</i>		$N(w-2)$	$N(4w-7)$	$\mathcal{O}(n^2)$
<i>BDD</i>		$2N(w-1)$	$4N(w-1)$	$\mathcal{O}(n^2)$
<i>Card</i>		$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$
<i>Adder</i>		$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$
<i>Product</i>		$N(2\sqrt{w} + \mathcal{O}(\sqrt[4]{w}))$	$N(2w + 4\sqrt{w} + \mathcal{O}(\sqrt[4]{w}))$	$\mathcal{O}(n^2)$
<i>Duplex</i>	Encode set of AMO constraints	$4M(w-1)$	$13Mw - 14M - 3w + 2$	$\mathcal{O}(n)$
$SCL_{\leq 1}$		$(2M-2)(w-2)$	$8Mw - 14M - 7w + 13$	$\mathcal{O}(n)$

Table 2 presents a summary of the evaluation results for various encoding techniques used to encode Ladder AMO constraints across n variables with a width of w . The comparison is based on several criteria, including the number of required auxiliary variables needed, the total number of generated clauses, and the complexity of the clause space. This evaluation incorporates the study by Fazekas et al. [14] as well as our proposed $SCL_{\leq k}$ encoding for the case when $k = 1$.

To represent AMO constraints, the *Cardinality Network* (*Card*) encoding selects $k = 2$ and generates two outputs, i.e. c_1 and c_2 . Then, the output c_2 is imposed to be *false*, ensuring that no more than one input variable can be assigned *true*. As a result, the number of auxiliary variables, required clauses, and the overall clause complexity of the *Card* encoding is $\mathcal{O}(n^2 \log_2^2 2) = \mathcal{O}(n^2)$.

The *Product* encoding is based on arranging variables in a two-dimensional coordinate system. The values along each axis are treated as additional variables and are then organized into a smaller two-dimensional coordinate system. This process is recursively applied until further decomposition into smaller coordinate systems is no longer possible. Through this representation technique, *Product* encoding utilizes $N(2\sqrt{w} + \mathcal{O}\sqrt[4]{w})$ auxiliary variables and $N(2w + 4\sqrt{w} + \mathcal{O}\sqrt[4]{w})$ clauses to encode a Ladder AMO constraint.

The *Duplex* encoding employs the decomposition technique and encodes each window separately by generating two *BDDs* with distinct variable arrangements. After that, these constructed windows are bonded together to maintain the AMO property of the original constraints. As presented in Section 4.2, the *Duplex* encoding requires approximately $13Mw - 14M - 3w + 2$ clauses and $4M(w - 1)$ auxiliary variables.

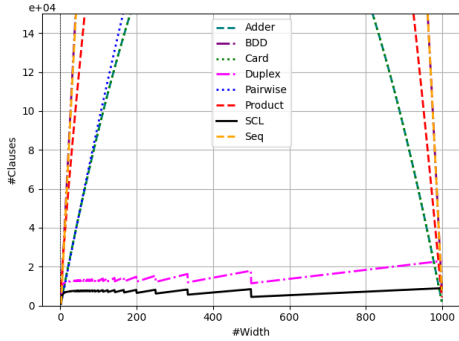


Fig. 10: Comparison of the number of clauses of a Ladder AMO constraint with parameters set to $n = 1000$ and $w \in (1, 1000)$.

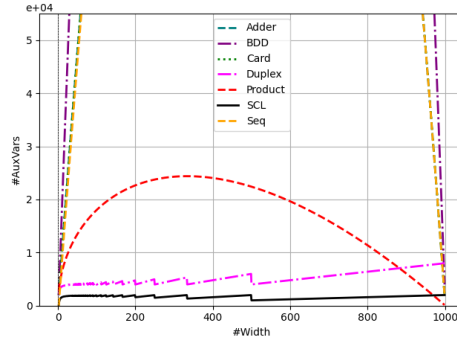


Fig. 11: Comparison of the number of auxiliary variables of a Ladder AMO constraint with parameters set to $n = 1000$ and $w \in (1, 1000)$.

The analysis presented above underscores the efficiency of our proposed encoding. Like the $SCL_{\leq k}$ encoding, the $SCL_{\leq 1}$ encoding outperforms several alternatives, including *Pairwise*, *Seq*, *BDD*, *Card*, and *Adder* encodings. As detailed in Table 2, the complexity of $SCL_{\leq 1}$ is $\mathcal{O}(n)$, which is also superior to $\mathcal{O}(n^2)$ of the *Product*

encoding. Furthermore, among linear complexity encodings, $SCL_{\leq 1}$ uses roughly half the number of variables and clauses compared to the *Duplex* encoding. To distinctly showcase the outstanding performance of our encoding, we present Figures 10 and 11, which illustrate the performance efficiency of the $SCL_{\leq 1}$ encoding. Figure 10 depicts the number of clauses, while Figure 11 illustrates the number of auxiliary variables required to encode a set of Ladder AMO constraints for $n = 1000$ and $w \in (1, 1000)$, based on the calculations presented in Table 2.

5 SAT solving for the Anti-bandwidth problem

The Anti-bandwidth problem (ABP) is an NP-hard problem with applications in various scheduling scenarios [25], including radio frequency assignment [26], obnoxious facility location [27], and map coloring [28]. In this section, we present SAT formulae, including the Ladder AMO constraints, for solving the ABP.

5.1 Problem definition

Let $G = (V, E)$ be a graph where V and E denote the sets of vertices and edges, respectively. A labeling function f is a bijection from V to $\{1, 2, \dots, |V|\}$, assigning a unique label to each vertex in V . The anti-bandwidth $AB_f(i)$ of a vertex $i \in V$ under the labeling function f is defined as the minimum absolute difference between the label of i and the labels of its adjacent vertices:

$$AB_f(i) = \min\{|f(i) - f(i')| : \{i, i'\} \in E\}$$

The anti-bandwidth of G under the labeling function f , denoted as $AB_f(G)$, is the minimal value among the $AB_f(i)$ values:

$$AB_f(G) = \min\{AB_f(i) : i \in V\}$$

The Anti-bandwidth Problem aims to find the maximum anti-bandwidth value of G , denoted as $AB(G)$, over all feasible labeling functions f .

$$AB(G) = \max AB_f(G)$$

5.2 ILP model for ABP

The ABP is expressible as an Integer Linear Programming (ILP) [10] model with binary (0–1) variables. By defining x_i^l to be a boolean variable that holds *true* if vertex i is assigned label l , and *false* otherwise, this model employs two formulas, (VERTICES) and (LABELS), to guarantee that every vertex in the graph receives a unique label. Specifically, equation (VERTICES) ensures that each label is allocated to exactly one vertex, while equation (LABELS) guarantees that each vertex is given precisely one label. Combining these two equations preserves the one-to-one relationship between vertices and labels.

$$\sum_{i \in V} x_i^l = 1 \quad \forall l \in \{1, \dots, |V|\} \quad (\text{VERTICES})$$

$$\sum_{l=1}^{|V|} x_i^l = 1 \quad \forall i \in V \quad (\text{LABELS})$$

Based on this approach, Markus Sinnl [10] proposed an iterative formulation, framed as a feasibility problem, to answer the question: “Does there exist a solution with $AB(G) \geq k + 1$?”. This formulation includes both formulas (VERTICES) and (LABELS), along with equation (OBJ-k), which ensures that the difference between the labels of adjacent vertices i and i' is strictly greater than k whenever $k + 1$ is a feasible anti-bandwidth.

$$\sum_{l'=l_2}^{l_2+k} (x_i^{l'} + x_{i'}^{l'}) \leq 1 \quad \forall \{i, i'\} \in E, \forall 1 \leq l_2 \leq |V| - k \quad (\text{OBJ-k})$$

For the ABP problem with 10 vertices ($|V| = 10$) and $k = 3$, the constraints given by equation (OBJ-k) for any pair of vertices i and i' are illustrated in Figure 12. These constraints ensure that the labels l_i and $l_{i'}$ of vertices i and i' , respectively, always satisfy the condition $|l_i - l_{i'}| \leq k + 1 = 4$. As an example, if $l_i = 4$, meaning x_i^4 holds *true*, then to prevent the set of AMO constraints (in Figure 12) from being violated, $x_{i'}^1, x_{i'}^2, x_{i'}^3, x_{i'}^4, x_{i'}^5, x_{i'}^6$, and $x_{i'}^7$ must all be *false*, which implies that $l_{i'} \notin \{1, 2, 3, 4, 5, 6, 7\}$. The remaining possible labels for vertex i' are $\{8, 9, 10\}$, ensuring that the label distance condition between vertices is maintained.

$$\begin{aligned} x_i^1 + x_i^2 + x_i^3 + x_i^4 + x_{i'}^1 + x_{i'}^2 + x_{i'}^3 + x_{i'}^4 &\leq 1 \\ x_i^2 + x_i^3 + x_i^4 + x_i^5 + x_{i'}^2 + x_{i'}^3 + x_{i'}^4 + x_{i'}^5 &\leq 1 \\ x_i^3 + x_i^4 + x_i^5 + x_i^6 + x_{i'}^3 + x_{i'}^4 + x_{i'}^5 + x_{i'}^6 &\leq 1 \\ x_i^4 + x_i^5 + x_i^6 + x_i^7 + x_{i'}^4 + x_{i'}^5 + x_{i'}^6 + x_{i'}^7 &\leq 1 \\ x_i^5 + x_i^6 + x_i^7 + x_i^8 + x_{i'}^5 + x_{i'}^6 + x_{i'}^7 + x_{i'}^8 &\leq 1 \\ x_i^6 + x_i^7 + x_i^8 + x_i^9 + x_{i'}^6 + x_{i'}^7 + x_{i'}^8 + x_{i'}^9 &\leq 1 \\ x_i^7 + x_i^8 + x_i^9 + x_i^{10} + x_{i'}^7 + x_{i'}^8 + x_{i'}^9 + x_{i'}^{10} &\leq 1 \end{aligned}$$

Fig. 12: OBJ-k constraint of $\{i, i'\}$ with $|V| = 10$ and $k = 3$.

5.3 SAT implementation of ABP constraints

5.3.1 Decomposition of $OBJ - k$ constraints

Encoding the equation (OBJ-k) requires a total of $|E|$ separate constraints, each corresponding to an edge in the set E . However, in most graphs, the number of edges is significantly greater than the number of vertices. To optimize the encoding of (OBJ-k), Fazekas et al. [14] employ Lemma 1 to decompose the constraints of (OBJ-k) as follows:

$$\sum_{l'=l_2}^{l_2+k} (x_i^{l'} + x_{i'}^{l'}) \leq 1$$

Lemma 1
 $\equiv$

$$\sum_{l'=l_2}^{l_2+k} x_i^{l'} \leq 1 \wedge \sum_{l'=l_2}^{l_2+k} x_{i'}^{l'} \leq 1 \wedge \left(\sum_{l'=l_2}^{l_2+k} x_i^{l'} \leq 0 \vee \sum_{l'=l_2}^{l_2+k} x_{i'}^{l'} \leq 0 \right)$$

Using this decomposition technique, the constraints from equation (OBJ-k) for any edge $\{x_i, x_{i'}\}$ can be represented through the corresponding vertex-level constraints of x_i and $x_{i'}$. These vertex-level constraints can be reused to represent multiple edges, which reduces the total number of constraints needed from $|E|$ to $|V|$.

In particular, each AMO constraint in equation (OBJ-k) is decomposed into two sub-AMO constraints, each involving only the variables associated with vertex i or vertex i' (see Figure 13 for an illustration). By decomposing all AMO constraints in (OBJ-k), the obtained sub-AMO constraints form two Ladder AMO constraints, as presented in Figures 14 and 15. These Ladder constraints can be directly encoded using the proposed $SCL_{\leq 1}$ encoding.

Additionally, the decomposition of (OBJ-k) uses disjunctions as connecting clauses to connect vertices i and i' , thereby ensuring that the AMO property is maintained for the (OBJ-k) constraint over the edge $\{i, i'\}$ (see Figure 16). These connecting clauses can be encoded using register bits required by the $SCL_{\leq 1}$ encoding of the corresponding (OBJ-k) constraints, in accordance with Proposition 2.

Proposition 2. *For any integer number i s.t. $1 \leq i < n$, the constraint*

$$x_1 + x_2 + \dots + x_n \leq 0$$

holds if and only if the following constraint holds:

$$(x_1 + \dots + x_i \leq 0) \wedge (x_{i+1} + \dots + x_n \leq 0).$$

For instance, the decomposition of $x_i^2 + x_i^3 + x_i^4 + x_i^5 + x_{i'}^2 + x_{i'}^3 + x_{i'}^4 + x_{i'}^5 \leq 1$ utilizes $x_i^2 + x_i^3 + x_i^4 + x_i^5 \leq 0 \vee x_{i'}^2 + x_{i'}^3 + x_{i'}^4 + x_{i'}^5 \leq 0$, as presented in Figure 13. Applying Proposition 2, $x_i^2 + x_i^3 + x_i^4 + x_i^5 \leq 0$ (resp. $x_{i'}^2 + x_{i'}^3 + x_{i'}^4 + x_{i'}^5 \leq 0$) can be further decomposed into $x_i^2 + x_i^3 + x_i^4 \leq 0 \wedge x_i^5 \leq 0$ (resp. $x_{i'}^2 + x_{i'}^3 + x_{i'}^4 \leq 0 \wedge x_{i'}^5 \leq 0$). These clauses can then be substituted with the corresponding SC register bits, as discussed in Section 3.2.

$$\begin{aligned}
& x_i^2 + x_i^3 + x_i^4 + x_i^5 + x_{i'}^2 + x_{i'}^3 + x_{i'}^4 + x_{i'}^5 \leq 1 \\
& \equiv \\
& (x_i^2 + x_i^3 + x_i^4 + x_i^5 \leq 1) \wedge (x_{i'}^2 + x_{i'}^3 + x_{i'}^4 + x_{i'}^5 \leq 1) \\
& \wedge (x_i^2 + x_i^3 + x_i^4 + x_i^5 \leq 0 \vee x_{i'}^2 + x_{i'}^3 + x_{i'}^4 + x_{i'}^5 \leq 0)
\end{aligned}$$

Fig. 13: Decomposition of the second AMO constraint in (OBJ-k) constraint of edge $\{i, i'\}$ with $|V| = 10$ and $k = 3$.

$$\begin{array}{ll}
x_i^1 + x_i^2 + x_i^3 + x_i^4 \leq 1 & x_{i'}^1 + x_{i'}^2 + x_{i'}^3 + x_{i'}^4 \leq 1 \\
x_i^2 + x_i^3 + x_i^4 + x_i^5 \leq 1 & x_{i'}^2 + x_{i'}^3 + x_{i'}^4 + x_{i'}^5 \leq 1 \\
x_i^3 + x_i^4 + x_i^5 + x_i^6 \leq 1 & x_{i'}^3 + x_{i'}^4 + x_{i'}^5 + x_{i'}^6 \leq 1 \\
x_i^4 + x_i^5 + x_i^6 + x_i^7 \leq 1 & x_{i'}^4 + x_{i'}^5 + x_{i'}^6 + x_{i'}^7 \leq 1 \\
x_i^5 + x_i^6 + x_i^7 + x_i^8 \leq 1 & x_{i'}^5 + x_{i'}^6 + x_{i'}^7 + x_{i'}^8 \leq 1 \\
x_i^6 + x_i^7 + x_i^8 + x_i^9 \leq 1 & x_{i'}^6 + x_{i'}^7 + x_{i'}^8 + x_{i'}^9 \leq 1 \\
x_i^7 + x_i^8 + x_i^9 + x_i^{10} \leq 1 & x_{i'}^7 + x_{i'}^8 + x_{i'}^9 + x_{i'}^{10} \leq 1
\end{array}$$

Fig. 14: Ladder AMO constraint associated with vertex i with $|V| = 10$ and $k = 3$.

Fig. 15: Ladder AMO constraint associated with vertex i' with $|V| = 10$ and $k = 3$.

$$\begin{array}{ll}
x_i^1 + x_i^2 + x_i^3 + x_i^4 \leq 0 & \vee \quad x_{i'}^1 + x_{i'}^2 + x_{i'}^3 + x_{i'}^4 \leq 0 \\
x_i^2 + x_i^3 + x_i^4 + x_i^5 \leq 0 & \vee \quad x_{i'}^2 + x_{i'}^3 + x_{i'}^4 + x_{i'}^5 \leq 0 \\
x_i^3 + x_i^4 + x_i^5 + x_i^6 \leq 0 & \vee \quad x_{i'}^3 + x_{i'}^4 + x_{i'}^5 + x_{i'}^6 \leq 0 \\
x_i^4 + x_i^5 + x_i^6 + x_i^7 \leq 0 & \vee \quad x_{i'}^4 + x_{i'}^5 + x_{i'}^6 + x_{i'}^7 \leq 0 \\
x_i^5 + x_i^6 + x_i^7 + x_i^8 \leq 0 & \vee \quad x_{i'}^5 + x_{i'}^6 + x_{i'}^7 + x_{i'}^8 \leq 0 \\
x_i^6 + x_i^7 + x_i^8 + x_i^9 \leq 0 & \vee \quad x_{i'}^6 + x_{i'}^7 + x_{i'}^8 + x_{i'}^9 \leq 0 \\
x_i^7 + x_i^8 + x_i^9 + x_i^{10} \leq 0 & \vee \quad x_{i'}^7 + x_{i'}^8 + x_{i'}^9 + x_{i'}^{10} \leq 0
\end{array}$$

Fig. 16: Connecting clauses obtained from the decomposition of (OBJ-k) constraint of edge $\{i, i'\}$ with $|V| = 10$ and $k = 3$.

To enhance our approach, we implement the constraint (LABELS) by utilizing auxiliary variables when constructing $SCL_{\leq 1}$ for each vertex in the graph. This strategy improves the optimization of the generated clauses, providing a more efficient alternative to encoding with x_i^l for the constraint (LABELS). The remaining (VERTICES) constraint is applied with exactly one encoding in SAT using a combination of *Product* and *Seq* encodings.

5.3.2 Symmetry breaking

Referring to Markus Sinnl [10], for every labeling f of an n -vertex graph, there exists a reversed labeling f' , where each label in f' is computed from its corresponding label in f using the following linear transformation:

$$f' = n + 1 - f.$$

This transformation ensures a one-to-one correspondence between the values in f and those in f' . Consequently, if f satisfies (*VERTICES*) and (*LABELS*) constraints, then f' also satisfies them. Moreover, this transformation also ensures that f' maintains the same anti-bandwidth as f , which is proven as follows.

$$\begin{aligned} |f'(i) - f'(i')| &= |(n + 1 - f(i)) - (n + 1 - f(i'))| \\ &= |f(i') - f(i)| \end{aligned}$$

Based on this observation, we apply the symmetry breaking technique [29] to reduce the search space. In our implementation of ABP, we employ symmetry breaking at one selected node using two different configurations: the first node and the highest-degree node.

6 SAT solving for the Nurse Rostering problem

6.1 Problem definition and ILP model

The Nurse Rostering Problem (NRP) aims to construct a work schedule for one or more nurses over a planning horizon of d days. On each day, a nurse may be assigned to a day shift (D), an evening shift (E), or a night shift (N). If the nurse is not assigned to any shift (D, E, or N) on a day, the nurse has an off-day (O). The schedule must satisfy a set of constraints related to work frequency and rest periods in order to ensure nurses' well-being and workability.

This study considers the NRP variant used by Bergman et al. [5], which focuses on designing a schedule for a single nurse, subject to the scheduling requirements listed in Table 3. Although this variant focuses on a single nurse, it captures the essential workload distributions and restrictions that also arise in the multi-nurse problem.

Let $x_{i,j}$ be a boolean variable that denotes whether the nurse is assigned to shift j on day i . More precisely, if the nurse is assigned to shift j on day i , then $x_{i,j} = \text{true}$. Otherwise, if the nurse is not assigned to shift j on day i , then $x_{i,j} = \text{false}$. Additionally, $x_{i,O} = \text{true}$ indicates that the nurse has an off-day on day i . Otherwise, the nurse must be assigned to exactly one of the three working shifts on day i . Since the nurse either has to work exactly one shift per day or has an off-day, the assignment variables must satisfy

$$\sum_{j \in \{D, E, N, O\}} x_{i,j} = 1 \quad \forall i = 1, \dots, d.$$

Table 4 presents the ILP model to construct the schedule that satisfies the requirements outlined in Table 3. In this table, each schedule requirement, except the first one, is expressed as a sequence of cardinality constraints.

Table 3: Schedule requirements of NRP [5].

1.	At most 1 work shift every day
2.	At least 20 work shifts every 28 days
3.	At least 4 off-days every 14 days
4.	At least 1 night shift every 14 days
5.	At most 4 night shifts every 14 days
6.	At least 4 evening shifts every 14 days
7.	At most 8 evening shifts every 14 days
8.	Nights shifts is not on consecutive days
9.	At least 2 evening/night shifts every 7 days
10.	At most 4 evening/night shifts every 7 days
11.	At most 6 work shifts every 7 days

Table 4: ILP model of NRP.

1.	$\sum_{j \in \{D,E,N,O\}} x_{i,j} = 1$	$\forall i \in 1, \dots, d$
2.	$\sum_{i'=i}^{i+27} \neg x_{i',O} \geq 20$	$\forall i \in 1, \dots, d-27$
3.	$\sum_{i'=i}^{i+13} x_{i',O} \geq 4$	$\forall i \in 1, \dots, d-13$
4.	$\sum_{i'=i}^{i+13} x_{i',N} \geq 1$	$\forall i \in 1, \dots, d-13$
5.	$\sum_{i'=i}^{i+13} x_{i',N} \leq 4$	$\forall i \in 1, \dots, d-13$
6.	$\sum_{i'=i}^{i+13} x_{i',E} \geq 4$	$\forall i \in 1, \dots, d-13$
7.	$\sum_{i'=i}^{i+13} x_{i',E} \leq 8$	$\forall i \in 1, \dots, d-13$
8.	$\sum_{i'=i}^{i+1} x_{i',N} \leq 1$	$\forall i \in 1, \dots, d-1$
9.	$\sum_{i'=i}^{i+6} x_{i',E} + x_{i',N} \geq 2$	$\forall i \in 1, \dots, d-6$
10.	$\sum_{i'=i}^{i+6} x_{i',E} + x_{i',N} \leq 4$	$\forall i \in 1, \dots, d-6$
11.	$\sum_{i'=i}^{i+6} \neg x_{i',O} \leq 6$	$\forall i \in 1, \dots, d-6$

6.2 SAT implementation of NRP constraints

The ILP model in Table 4 primarily consists of sequence constraints with ladder structure over the planning horizon. Some of them are naturally the Ladder AMK constraint with $k \geq 2$, which can be encoded by applying the proposed encoding directly. However, several of these constraints are of the At-Least-K type with $k \geq 2$, such as constraints (2), (3), (6), and (9). Therefore, to leverage the advantages of our proposed encoding, we systematically transform these constraints into equivalent Ladder At-Most-K constraints.

In particular, constraint (2) requires at least 20 working days in every 28 consecutive days. Since each day is either a working day or an off-day, this constraint is equivalent to limiting the number of off-days to at most 8 in every such consecutive days.

$$\sum_{i'=i}^{i+27} \neg x_{i',O} \geq 20 \equiv \sum_{i'=i}^{i+27} x_{i',O} \leq 8$$

Similarly, constraint (3), which requires at least 4 off-days every 14 days, can be reformulated as restricting the number of working days to at most 10.

$$\sum_{i'=i}^{i+13} x_{i',O} \geq 4 \equiv \sum_{i'=i}^{i+13} \neg x_{i',O} \leq 10$$

Constraint (6), imposing at least 4 evening shifts every 14 days, can be reformulated as restricting the number of days the nurse does not work in the evening to at most 10.

$$\sum_{i'=i}^{i+13} x_{i',E} \geq 4 \equiv \sum_{i'=i}^{i+13} \neg x_{i',E} \leq 10$$

Since the nurse has to work at most 1 shift every day, constraint (9), which states that the nurse has to work at least 2 evening or night shifts every 7 days, is equivalent to imposing that the total number of day shifts and off-days is at most 5 every 7 days.

$$\sum_{i'=i}^{i+6} x_{i',E} + x_{i',N} \geq 2 \equiv \sum_{i'=i}^{i+6} x_{i',D} + x_{i',O} \leq 5$$

7 Experimental Evaluation

We conducted three experiments to evaluate the effectiveness of our proposed SAT encoding *SCL*. The first experiment compares the solving times of *SCL* against other SAT encodings when addressing single Ladder AMO constraints ($k = 1$) and Ladder AMK constraints ($k > 1$). The second and third experiments, respectively, focus on evaluating $SCL_{\leq 1}$ and $SCL_{\leq k}$ in solving the Anti-bandwidth Problem and Nurse Rostering Problem. These three experiments were performed on a cluster provided by the *Google Cloud Platform*¹ featuring 8 vCPUs, 4 cores, and 64GB of memory, running the *Debian GNU/Linux 12 operating system*. The SAT solver used was CaDiCaL version 1.2.1 [30].

7.1 Single Ladder AMK constraint

The number of variables and clauses generated by the $SCL_{\leq 1}$ and $SCL_{\leq k}$ encodings, compared to other SAT encodings, is detailed in Tables 1 and 2 of the paper. In this experiment, we aim to evaluate the solving time of *SCL* relative to other SAT encodings when given a Ladder AMK constraint as input, while varying the number of variables and the width w .

7.1.1 Solving time of the Ladder AMO constraint

The experimental dataset for the Ladder AMO constraint consists of a fixed number of variables, specifically $n = 1000$. The width w ranges from 5 to 995 in increments of 5 (i.e., 5, 10, 15, ..., 995). The different SAT encodings compared to the $SCL_{\leq 1}$ are detailed in Table 2. These encodings include *Pairwise*, *Sequential Counter (Seq)*, *Binary Decision Diagrams (BDD)*, *Cardinality Network (Card)*, *Adder Network (Adder)*, *Product*, and *Duplex*. Figures 17 and 18 illustrate the solving times for the various encodings.

Among the top three encodings, as shown in Figure 18, $SCL_{\leq 1}$ emerged as the most effective, surpassing *Duplex*, which was previously recognized as an efficient encoding

¹<https://console.cloud.google.com/compute>

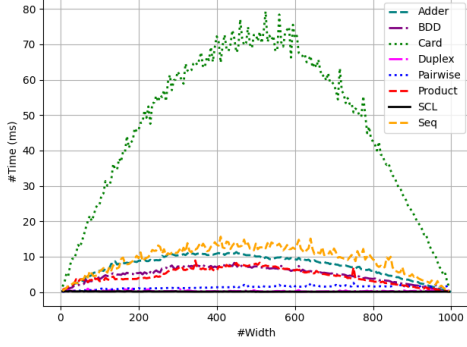


Fig. 17: Solving time of various encodings on the Ladder AMO constraints with $n = 1000$ and $w \in \{5, 10, 15, \dots, 995\}$.

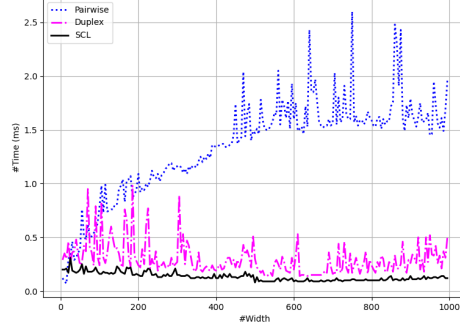


Fig. 18: Solving time of the top three encodings on the Ladder AMO constraints with $n = 1000$ and $w \in \{5, 10, 15, \dots, 995\}$.

[14]. Overall, the results indicate that, in most cases, the $SCL_{\leq 1}$ encoding significantly outperforms all other SAT encodings in terms of time efficiency. Additionally, the effectiveness of the generated clauses and the necessary auxiliary variables for $SCL_{\leq 1}$ are discussed earlier in Section 4.3.

7.1.2 Solving time of the Ladder AMK constraint

To compare the solving times of $SCL_{\leq k}$ with other SAT encodings, we employed a dataset of Ladder AMK constraints that included n variables and a width w . The values for n were chosen from the range $\{100, 200, 300, \dots, 5000\}$, while the values for w were taken from the set $\{10, 20, 50, 100\}$. We define k as $k = \frac{1}{5}w$ (which means $k \in \{2, 4, 10, 20\}$). Each instance was allowed to run for a maximum of 600 seconds. The SAT encodings compared to $SCL_{\leq k}$ are detailed in Table 1, which includes *Pairwise*, *Seq*, *BDD*, *Adder*, and *Card*. *Product* encoding is excluded from this experiment, as it is less efficient for solving AMK constraints.

The solving times of SAT encodings for four values of k are illustrated in Figures 19, 20, 21, and 22. The encoding $SCL_{\leq k}$ achieves the fastest solving times for $\{w, k\} = \{20, 4\}$, $\{50, 10\}$, and $\{100, 20\}$, and it ranks second after the *Pairwise* encoding for $\{w, k\} = \{10, 2\}$. Compared to *Seq*, *BDD*, *Card*, and *Adder*, $SCL_{\leq k}$ consistently demonstrates superior performance regarding solving time.

For instance, when $n = 5000$, the solving time for $SCL_{\leq k}$ is approximately one-third of the time required by the second fastest encoding, *BDD*. Specifically, for $\{w, k\} = \{20, 4\}$, $SCL_{\leq k}$ solves the problem in 6.6 milliseconds, while *BDD* takes 18.1 milliseconds. For $\{w, k\} = \{50, 10\}$, $SCL_{\leq k}$ takes 23 milliseconds compared to *BDD*'s 106 milliseconds. For $\{w, k\} = \{100, 20\}$, $SCL_{\leq k}$ finishes in 46.6 milliseconds, while *BDD* takes 298.4 milliseconds. The *Pairwise* encoding is the fastest for $\{w, k\} = \{10, 2\}$ but begins to slow down at $\{w, k\} = \{20, 4\}$. It times out for all input sizes n within the 600-second limit when solving for $\{w, k\} = \{50, 10\}$ and $\{100, 20\}$.

² *Pairwise* encoding times out for all input sizes n within the 600-second limit.

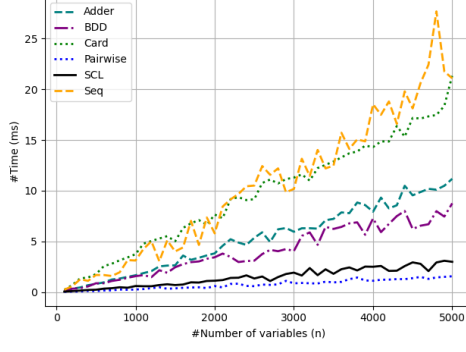


Fig. 19: Solving time of Ladder AMK constraints with $w = 10$ and $k = 2$.

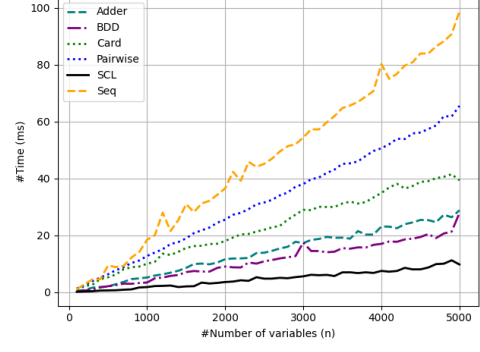


Fig. 20: Solving time of Ladder AMK constraints with $w = 20$ and $k = 4$.

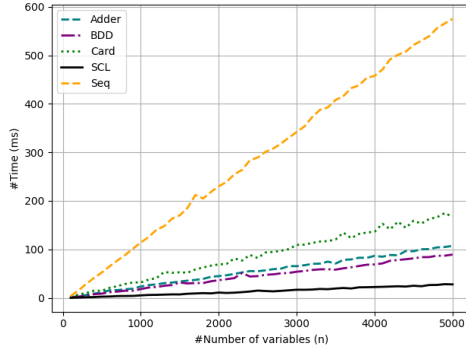


Fig. 21: Solving time of Ladder AMK constraints² with $w = 50$ and $k = 10$.

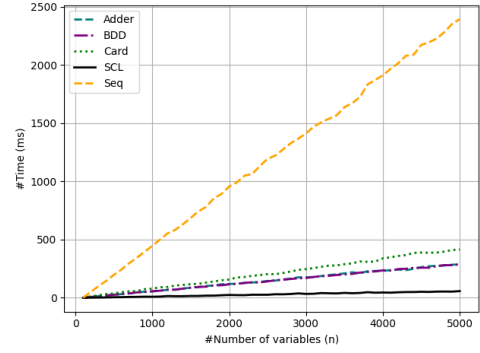


Fig. 22: Solving time of Ladder AMK constraints² with $w = 100$ and $k = 20$.

7.2 Computation Results for the Anti-bandwidth Problem

7.2.1 Datasets and Implementation Details

The second framework evaluates the effectiveness of the proposed encoding in addressing the Anti-bandwidth Problem. It utilizes a dataset consisting of 24 matrices from the Harwell-Boeing Sparse Matrix Collection [31]. This collection includes 12 small instances and 12 large instances. Detailed information about these 24 matrices can be found in Table 5, which lists the names of the matrices, the number of vertices $|V|$, and the number of edges $|E|$. Additionally, the corresponding lower and upper bounds (LB and UB) are provided by Fazekas et al. [14].

In this framework, we implemented the $SCL_{\leq 1}$ encoding alongside four SAT-based encodings: *Product*, *Pairwise*, *Sequential Counter*, and *Duplex*. Throughout the implementation, we followed a consistent sequential process. We began by setting the lower bound (LB) as the starting width for the Ladder AMO constraints. If the SAT solver

Table 5: Harwell-Boeing Sparse Matrix benchmark.

Small instances					Large instances				
Instance	V	E	LB	UB	Instance	V	E	LB	UB
A-pores_1	30	103	6	8	M-bcsstk06	420	3720	28	72
B-ibm32	32	90	9	9	N-bcsstk07	420	3720	28	72
C-bcspwr01	39	46	16	17	O-impcol_d	425	1267	91	173
D-bcsstk01	48	176	8	9	P-can_445	445	1682	78	120
E-bcspwr02	49	59	21	22	Q-494_bus	494	586	219	246
F-curtis54	54	124	12	13	R-dwt_503	503	2762	46	71
G-will57	57	127	12	14	S-sherman4	546	1341	256	272
H-impcol_b	59	281	8	8	T-dwt_592	592	2256	103	150
I-ash85	85	219	19	27	U-662_bus	662	906	219	220
J-nos4	100	247	32	40	V-nos6	675	1290	326	337
K-dwt_234	117	162	46	58	W-685_bus	685	1282	136	136
L-bcspwr03	118	179	39	39	X-can_715	715	2975	112	142

returns a satisfiable (SAT) result, we increase the width by 1 and restart the process. Conversely, if the solver returns unsatisfiable (UNSAT) or reaches the upper bound (UB), it indicates that we have found the optimal solution, and the iteration process concludes.

Constraint Programming (CP) and Mixed Integer Programming (MIP) are included in this experiment, which are $F_e(k)$, *CPLEX*, *CP-MZ-Chuffed*, *CP-SAT*, and *Gurobi*. In our experiment, the $F_e(k)$ encoding leverages the MIP APIs from IBM ILOG CPLEX Optimization Studio³ version 20.1, a robust framework that integrates constraint propagation with advanced optimization techniques for efficient problem-solving. The *CPLEX* approach utilizes the CP APIs of IBM ILOG CPLEX Optimization Studio version 22.1.1. The *CP-MZ-Chuffed* method employs the MiniZinc language [32] version 2.8.6 and incorporates Chuffed solver⁴ version 0.13.2, which excels in lazy clause generation and is particularly effective for combinatorial problems. The *CP-SAT* approach relies on the CP-SAT Solver, a versatile solver designed for constraint satisfaction and optimization, leveraging multi-processing and SAT-based techniques to handle complex constraints, which is provided by Google OR-Tools⁵ version 9.11.4210. Finally, the *Gurobi* encoding harnesses the Gurobi Optimizer⁶ version 11.0.3, a powerful tool for mathematical optimization, known for its efficiency in handling large-scale constraint-based problems. Note that *CPLEX*, *Gurobi*, and *CP-SAT* are powerful CP and MIP solvers that have been under development for a long time.

In addition to the sequential implementation, referred to as SCL_{seq} , which incrementally solves for anti-bandwidth values from LB to UB, we also implemented a multi-process approach, referred to as SCL_{par} , with the aim of providing a fair comparison with state-of-the-art CP and MIP solvers that exploit multi-core architectures,

³<https://www.ibm.com/products/ilog-cplex-optimization-studio/cplex-optimizer>

⁴<https://github.com/chuffed/chuffed.git>

⁵<https://developers.google.com/optimization/cp/cp-solver>

⁶<https://www.gurobi.com/>

such as *CPLEX*, *Gurobi*, and *CP-SAT*. This implementation initializes multiple processes, with each process invoking a SAT solver to address a different anti-bandwidth. The multi-process approach is more efficient than the sequential method because if one process finds a solution for a higher anti-bandwidth, it can terminate other processes working on smaller anti-bandwidths, hence allows the search to concentrate on better values. In this experiment, SCL_{par} is executed with up to four parallel processes invoking the SAT solver.

7.2.2 Performance Evaluation

Table 6 summarizes our experimental results on 24 selected matrices, with a time limit of 1800 seconds and a memory limit of 30 GB applied to the *Product*, *Pairwise*, *Sequential Counter* (Seq), *Duplex*, SCL_{seq} , SCL_{par} , $F_e(k)$, *CPLEX*, *CP-MZ-Chuffed*, *CP-SAT*, and *Gurobi* approaches. For each problem, the table displays the best solution found by each method, along with the solving time (in seconds) and memory consumption (in MB). The best anti-bandwidth value is highlighted in bold, and the best solving time is underlined. If the computation time exceeds 1800 seconds or the memory usage surpasses 30 GB, a termination signal is triggered, resulting in the process stopping due to a timeout (TO) or memory overload (MO), respectively. In such cases, the highest solved anti-bandwidth is reported as the best result achieved by the technique. If the method fails to solve the problem using the initial lower bound (LB) value, the result is indicated with a “-” character.

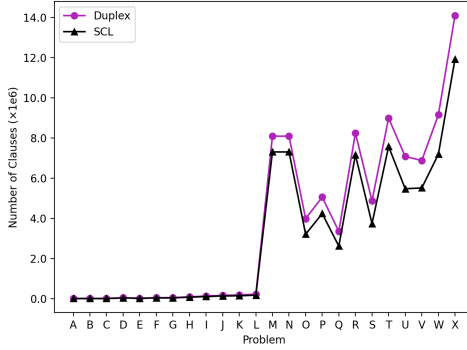


Fig. 23: Comparison of generated clauses between *SCL* and *Duplex*.

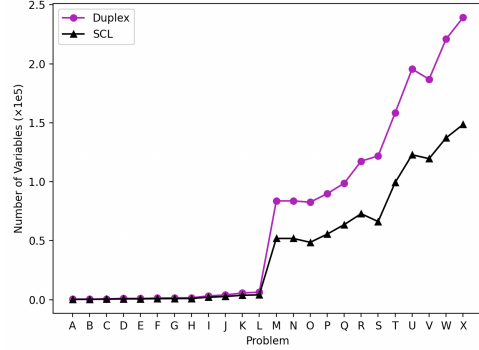


Fig. 24: Comparison of auxiliary variables between *SCL* and *Duplex*.

Figures 23 and 24 compare the number of generated clauses and auxiliary variables used in the ABP encoding across 24 instances. This evaluation considers both the *SCL* and *Duplex* encodings, with *Duplex* recognized as the best encoding according to [14]. Each instance is represented by the first letter of its name (e.g., *A* for *A-pores_1* and *X* for *X-can_715*). We analyzed the encodings at the maximum anti-bandwidth achievable by both encodings within specified time and memory limits. The results show that as the size of the instance increases, indicated by the increasing number

Table 6: ABP solving results of different approaches with TO = 1800s and MO = 30GB.

Instance	LB	UB	<i>Product</i>			<i>Pairwise</i>			<i>Seq</i>			<i>Duplex</i>			<i>SCL_{seq}</i>			<i>SCL_{par}</i>			<i>F_c(k)</i>			<i>CPLX</i>			<i>CP-MZ-Chuffed</i>			<i>CP-SAT</i>			<i>Gurobi</i>		
			Obj-k	Time(s)	MB	Obj-k	Time(s)	MB	Obj-k	Time(s)	MB	Obj-k	Time(s)	MB	Obj-k	Time(s)	MB	Obj-k	Time(s)	MB	Obj-k	Time(s)	MB	Obj-k	Time(s)	MB	Obj-k	Time(s)	MB	Obj-k	Time(s)	MB	Obj-k	Time(s)	MB
A-pores ₁	6	8	6	122	77.5	6	65.7	56.9	6	47.7	55.4	6	<u>3.4</u>	14.1	6	9.15	19.8	6	8.8	23.6	6	3.53	74.9	7	TO	579.3	6	4.75	42.3	6	5.17	144.9	6	9.84	84.9
B-ibm32	9	9	9	6.02	48.9	9	18.7	34.5	9	2.13	42.9	9	<u>0.19</u>	8.8	9	0.3	7.8	9	0.21	9.2	9	4.57	106.8	9	0.39	41.3	9	1.06	29.3	9	1.38	120.5	9	6.99	108.2
C-besprw01	16	17	17	5.25	41.4	17	13	40.1	17	20.7	58.2	17	1.11	11.2	17	4.19	18.6	17	3.44	16.8	17	3.59	97.5	17	0.67	41.0	17	<u>0.43</u>	28.6	17	1.46	126.1	17	5.1	104.6
D-besstk01	8	9	9	9.43	140.1	9	5.36	48.2	9	11.0	118.9	9	0.54	14.9	9	0.19	12.6	9	<u>0.13</u>	13.2	9	14.9	147.0	9	2.28	46.8	9	1.12	41.0	9	1.72	158.4	9	15.9	252.9
E-besprw02	21	22	21	8.79	83.5	21	9.22	45.8	21	9.91	86.1	21	2.15	14.9	21	1.52	11	21	<u>0.74</u>	19.3	21	17.8	170.2	21	15.83	51.5	21	0.8	31.5	21	3.83	172.6	21	14.3	198.5
F-curtis54	12	13	13	13.6	141.6	13	1.52	41.3	13	11.1	127	13	0.27	14.1	13	0.28	12.3	13	<u>0.13</u>	23.9	13	13.3	145.8	13	2.5	47.2	13	1.35	38.8	13	2.6	179.8	13	14.3	255
G-will57	12	14	13	16.6	147.9	13	11.9	73.2	13	23.2	133.8	13	<u>0.29</u>	14.4	13	0.47	13.9	13	0.44	26.5	13	18.5	175.6	13	14.18	51.5	13	1.09	39.9	13	2.16	176.2	13	21.5	271.3
H-impcol_b	8	8	8	4.38	177	8	0.38	52.8	8	5.22	147.7	8	0.26	21.0	8	0.23	16.8	8	<u>0.08</u>	4.8	8	3.12	167.4	8	0.32	44.7	8	0.86	55.5	8	2.06	190.2	8	8.17	262.5
I-ash85	19	27	22	TO	870.8	22	TO	843.6	23	TO	619.8	24	TO	272	24	TO	381	24	TO	496.6	22	TO	460.3	23	TO	288.2	23	TO	374.8	24	TO	1723	22	TO	1219
J-nos4	32	40	33	TO	1119.5	33	TO	1070.1	-	TO	1572.8	35	472	177	35	980	503	35	720.3	505.5	-	TO	927.0	35	TO	272.7	35	<u>384.4</u>	343.9	36	TO	1818	33	TO	1690
K-dwt ₂₃₄	46	58	49	TO	1373.3	51	TO	1192.6	48	TO	1110	51	TO	685	51	TO	530	52	TO	1377	50	TO	895.3	52	TO	243.8	47	TO	244.6	51	TO	1088	50	TO	784.4
L-besprw03	39	39	39	15.1	682.1	39	3.76	436.5	39	47.7	856.6	39	0.58	53	39	0.69	40.9	39	0.65	41	39	11.15	559.0	39	<u>0.25</u>	48.2	39	2.34	66.6	39	7.15	482.4	39	65.9	321.3
M-besstk06	28	72	-	-	MO	31	TO	22469	-	-	MO	35	TO	1726	35	TO	1398	35	TO	5076	-	TO	12207	30	TO	738.4	-	TO	3016	33	TO	5243	-	320.3	MO
N-besstk07	28	72	-	-	MO	31	TO	22470	-	-	MO	35	TO	1726	35	TO	1397	35	TO	5291	-	TO	12244	30	TO	737.9	-	TO	3016	33	TO	5250	-	304.1	MO
O-impcol_d	91	173	-	-	MO	96	TO	22681	-	-	MO	101	TO	1745	102	TO	1802	106	TO	5366	-	-	MO	121	TO	491.3	-	TO	1410	93	TO	5388	-	341	MO
P-can ₄₄₅	78	120	-	-	MO	-	TO	27109	-	-	MO	-	TO	2464	79	TO	2156	81	TO	9758	-	-	MO	79	TO	869.2	-	TO	1843	-	TO	5732	-	411.6	MO
Q-494 _{bus}	219	246	-	TO	25945	-	TO	29391	-	-	MO	-	TO	1443	220	TO	1301	220	TO	4713	-	TO	23775	220	TO	701.6	-	TO	763.8	-	316.8	MO	-	TO	18775
R-dwt ₅₀₃	46	71	-	-	MO	-	-	MO	-	-	MO	64	TO	1634	64	TO	1474	64	TO	4991	-	TO	10633	56	TO	717.6	-	TO	1950	49	TO	7347	-	424.5	MO
S-sherman4	256	272	-	-	MO	-	-	MO	-	-	MO	-	TO	1326	-	TO	1073	-	TO	4003	-	-	MO	-	TO	689.9	-	TO	888.8	-	TO	29192	-	1013	MO
T-dwt ₅₉₂	103	150	-	-	MO	-	-	MO	-	-	MO	-	TO	5357	104	TO	2619	104	TO	11445	-	-	MO	104	TO	1366	-	TO	1905	-	TO	9331	-	871.3	MO
U-662 _{bus}	219	220	-	-	MO	-	-	MO	-	-	MO	220	471	2498	220	144	1377	220	122.9	2600	-	-	MO	220	<u>3.15</u>	250.4	-	TO	1061	-	328	MO	-	851.3	MO
V-nos6	326	337	-	-	MO	-	-	MO	-	-	MO	-	TO	1836	-	TO	1532	-	TO	6317	-	-	MO	-	TO	967.9	-	TO	1168	-	1260	MO	-	1071	MO
W-685 _{bus}	136	136	-	-	MO	-	-	MO	-	-	MO	136	10.7	1431	136	7.57	1164	136	7.6	1162	-	-	MO	136	<u>1.95</u>	332.6	-	TO	1325	136	73.3	8223	-	855	MO
X-can ₇₁₅	112	142	-	-	MO	-	-	MO	-	-	MO	-	TO	3060	113	TO	4599	116	TO	10388	-	-	MO	113	TO	1380	-	TO	2442	-	TO	11514	-	1013	MO

of vertices $|V|$ and edges $|E|$, the SCL encoding requires fewer clauses and auxiliary variables than the $Duplex$ encoding. This finding highlights why SCL can outperform $Duplex$ in solving certain ABP instances, particularly in 12 large instances, and it also demonstrates more efficient memory usage.

When solving 12 small graph instances, the SCL encoding yields the best results compared to other methods. The performance of SCL_{seq} is competitive with $Duplex$ and outperforms the other SAT-based encodings. Compared to $CPLEX$, which is one of the top-performing methods among CP and MIP approaches, SCL_{par} finds the optimal anti-bandwidth values for the two instances $A-pores_1$ and $J-nos_4$, which are 6 and 35, respectively. In contrast, $CPLEX$ only proves feasible anti-bandwidths of 6 and 34 for those instances. Moreover, SCL_{par} also performs better on the $I-ash85$ instance. Compared to the other efficient CP method, $CP-SAT$, SCL_{par} delivers better results on the instances $J-nos_4$ and $K-dwt_{...234}$. MIP-based methods such as $Gurobi$ and $F_e(k)$ exhibit a weaker performance than SCL_{par} .

Table 7: ABP solving results of the five best approaches in 12 large instances with TO = 1800s and MO = 30GB.

Instance	LB	UB	<i>Duplex</i>			<i>SCL_{seq}</i>			<i>SCL_{par}</i>			<i>CPLEX</i>			<i>CP-SAT</i>		
			Obj-k	Time(s)	MB	Obj-k	Time(s)	MB	Obj-k	Time(s)	MB	Obj-k	Time(s)	MB	Obj-k	Time(s)	MB
M-bcsstk06	28	72	35	TO	1726	35	TO	1398	35	TO	5076	30	TO	738.4	33	TO	5243
N-bcsstk07	28	72	35	TO	1726	35	TO	1397	35	TO	5291	30	TO	737.9	33	TO	5250
O-impcol_d	91	173	101	TO	1745	102	TO	1802	106	TO	5366	121	TO	491.3	93	TO	5388
P-can_445	78	120	-	TO	2464	79	TO	2156	81	TO	9758	79	TO	869.2	-	TO	5732
Q-494_bus	219	246	-	TO	1443	220	TO	1301	220	TO	4713	220	TO	701.6	-	316.8	MO
R-dwt_503	46	71	64	TO	1634	64	TO	1474	64	TO	4991	56	TO	717.6	49	TO	7347
S-sherman4	256	272	-	TO	1326	-	TO	1073	-	TO	4003	-	TO	689.9	-	TO	29192
T-dwt_592	103	150	-	TO	5357	104	TO	2619	104	TO	11445	104	TO	1366	-	TO	9331
U-662_bus	219	220	220	471	2498	220	144	1377	220	122.9	2600	220	<u>3.15</u>	250.4	-	328	MO
V-nos6	326	337	-	TO	1836	-	TO	1532	-	TO	6317	-	TO	967.9	-	1260	MO
W-685_bus	136	136	136	10.7	1431	136	7.57	1164	136	7.6	1162	136	<u>1.95</u>	332.6	136	73.3	8223
X-can_715	112	142	-	TO	3060	113	TO	4599	116	TO	10388	113	TO	1380	-	TO	11514

Table 7 summarizes the experimental results on 12 large problem instances for the five best-performing approaches: $Duplex$, SCL_{seq} , SCL_{par} , $CPLEX$, and $CP-SAT$. Overall, SCL_{par} achieves the best performance among all methods, except for the $O-impcol_d$ instance, where $CPLEX$ performs significantly better. Compared to $CPLEX$, SCL_{par} outperforms it on five instances: $M-bcsstk06$, $N-bcsstk07$, $P-can_445$, $R-dwt_503$, and $X-can_715$. In the sequential setting, SCL_{seq} consistently matches or improves upon the results obtained by $Duplex$ across all 12 large instances, demonstrating that the advantage over $Duplex$ is inherent in the proposed SCL encoding. Building on this, SCL_{par} represents an improved variant of SCL_{seq} that further exploits parallel search to enhance performance, outperforming $Duplex$ on five instances, including $O-impcol_d$, $P-can_445$, $Q-494_bus$, $T-dwt_592$, and $X-can_715$. Notably, for $P-can_445$ and $X-can_715$, no other approach achieves results comparable to those of SCL_{par} .

7.3 Computation Results for the Nurse Rostering Problem

7.3.1 Datasets and Implementation Details

The third framework evaluates the performance of the SCL encoding on a Nurse Rostering Problem (NRP) variant used by Bergman et al. [5]. This NRP variant involves scheduling for a single nurse over a planning horizon ranging from 40 to 100 days, subject to 11 schedule requirements described in Section 6.

In this framework, we implemented the $SCL_{\leq k}$ alongside several SAT encoding techniques, including *BDD*, *Card*, *Adder*, *Pairwise*, *SCL*, and *Seq*. For the SAT-based approaches, constraints 1, 4, and 8 in the ILP model are encoded using *Pairwise* encoding, since they correspond to At-Least-One, At-Most-One, or Exactly-One constraints. For the remaining constraints of ALK type, particularly constraints (2), (3), (6), and (9), if a given SAT encoding cannot handle them directly, we first reformulate these constraints into equivalent AMK constraints, as described in Section 6.2.

We also compare $SCL_{\leq k}$ with state-of-the-art CP and MIP solvers, namely *CPLEX*, *CP-SAT*, and *Gurobi*. As in the experiments on the Anti-bandwidth Problem, the *CPLEX* approach leverages the CP APIs of IBM ILOG CPLEX Optimization Studio version 22.1.1, while *CP-SAT* is provided by Google OR-Tools version 9.11.4210, and *Gurobi* uses Gurobi Optimizer version 11.0.3.

7.3.2 Performance Evaluation

Since almost all approaches can solve the NRP dataset quickly, we execute all the approaches on each instance 10 times and report the average solving time and memory consumption. For each run, the time limit and memory limit are set to 600 seconds and 5 GB, respectively. The results, including both solving time and memory consumption, are summarized in Table 8 and also illustrated in Figures 25 and 26. Figure 25 presents the solving time, while Figure 26 reports the memory consumption.

Work period	40 days		60 days		80 days		100 days	
	Time(ms)	MB	Time(ms)	MB	Time(ms)	MB	Time(ms)	MB
<i>Adder</i>	192	9.6	302	15	419	19.3	535	26.4
<i>BDD</i>	49	8.9	94	15.2	145	23	187	28
<i>Card</i>	107	10.8	143	16.7	180	24.5	212	29.5
<i>Pairwise</i>	-	MO	-	MO	-	MO	-	MO
<i>SCL</i>	16	5.7	25	8.6	33	11.5	42	13.4
<i>Seq</i>	27	13.9	53	26	86	40.2	106	47.4
<i>CPLEX</i>	38	9.3	57	12.5	81	16	82	19.4
<i>CP-SAT</i>	59	17.1	83	19.1	140	26	148	29.9
<i>Gurobi</i>	18	15.4	27	14.8	41	18.4	59	16.4

Table 8: NRP solving results of different approaches with TO = 600s and MO = 5GB.

In terms of solving time, *SCL* encoding consistently outperforms other SAT-based encodings across all instances. Compared with the best-performing alternative SAT-based encoding, namely *Seq*, *SCL* solves all instances approximately twice as fast. For example, on the smallest instance (40 days), *SCL* finds a feasible schedule in 16 ms, whereas *Seq* requires 27 ms. On the largest instance (100 days), *SCL* requires only 42 ms, compared to 106 ms for *Seq*. These results demonstrate a substantial

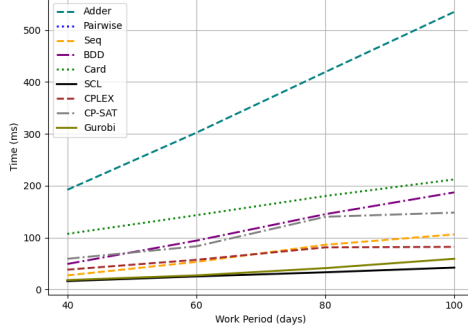


Fig. 25: NRP solving time of different approaches.

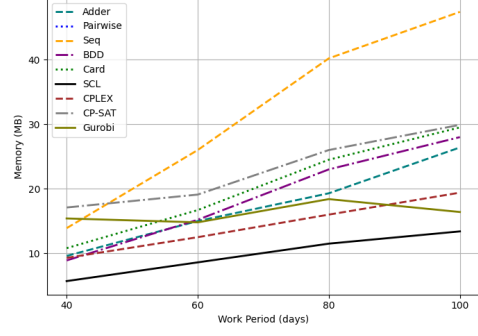


Fig. 26: NRP memory usage of different approaches.

runtime improvement of *SCL* encoding over existing SAT encodings that encode each cardinality constraint separately.

Importantly, this speed advantage is not achieved at the expense of memory usage. *SCL* encoding also maintains the lowest memory consumption among all SAT-based encodings. Its memory usage increases moderately from 5.7 MB at 40 days to 13.4 MB at 100 days. In comparison, *Adder*, *BDD* and *Card* require roughly twice as much memory, while *Seq* consumes approximately three times more. Besides, *Pairwise* encoding fails on all instances due to memory overflow, confirming its lack of scalability when encoding AMK and ALK constraints.

When compared with CP and MIP solvers, *SCL* encoding remains competitive in solving time while maintaining lower memory consumption, especially on medium and large instances. On the 40-day instance, *SCL* finds a feasible schedule in 16 ms, slightly faster than 18 ms of *Gurobi*. The performance gap widens as the problem size increases. On the 100-day instance, *SCL* requires only 42 ms, whereas *Gurobi* needs 59 ms, meaning that *SCL* solves the instance in roughly two-thirds of the time.

It is also worth noting that all the CP and MIP solvers used in this experiment run in parallel using default settings with 8 concurrent threads, whereas the SAT-based approaches operate in single-threaded mode. The fact that *SCL* achieves better performance despite this difference further highlights its efficiency for problems involving Ladder constraints.

8 Conclusion

SAT solving has emerged as a competitive and effective alternative to CP and MIP solvers for various combinatorial, scheduling, and OR problems. Alongside significant advancements in the capabilities of SAT solvers, the development of efficient encodings tailored to specific types of constraints can greatly enhance overall solution performance. Numerous studies have shown that optimizing SAT encoding can lead to substantial improvements, especially when replacing direct encoding techniques with more compact representations. These optimized encodings can reduce the number of auxiliary variables, generated clauses, and clause sizes.

In this paper, we introduced a novel SAT encoding for sets of AMK cardinality constraints arranged in a ladder shape. Rather than encoding each AMK constraint individually, our approach generates encodings for blocks that represent adjacent AMK constraints with the aim of encoding a set of AMK constraints simultaneously. To our knowledge, previous research in this area has mainly focused on using *BDD* to represent Ladder AMO constraints [14], and no proposed encodings exist for Ladder AMK constraints where $k > 1$. Compared to the *BDD*-based representation and other SAT-based encodings that encode each AMK constraint separately, the proposed encoding, built upon the *Sequential Counter* encoding, achieves significant improvements for Ladder AMO constraints and demonstrates effectiveness for Ladder AMK constraints.

We further applied the proposed encoding to address the Anti-bandwidth Problem and Nurse Rostering Problem. The experimental results show that our approach is not only competitive with other SAT-based encodings but also capable of outperforming state-of-the-art CP and MIP solvers such as *CPLEX*, *Gurobi*, and *CP-SAT*. These findings highlight the practical potential of the proposed encoding for real-world applications, such as Car Sequencing problem [1, 2], Employee Scheduling problem [7], and various other scheduling challenges featuring sequence constraints with ladder structures.

Finally, while this work focuses on a block-based *SC* encoding tailored to Ladder AMK constraints, exploring the suitability of alternative cardinality encodings for enabling intermediate variable reuse remains an interesting direction for future research. In particular, it would be valuable to systematically investigate how other encoding structures can be adapted or extended to better support sequence constraints, especially Ladder AMK constraints. Such an analysis could provide deeper insights into the interaction between encoding structure and constraint overlap, and may further enhance the performance of SAT-based approaches for Ladder AMK constraints, as well as sequence constraints.

Acknowledgements. We sincerely thank the reviewers for their careful reading of the manuscript and constructive comments. Their precise and valuable suggestions have significantly improved the clarity, quality, and technical soundness of this work.

Author Contributions. Hieu Truong Xuan: conceptualization, methodology, formal analysis, investigation, data curation, software, visualization, writing - original draft, review & editing;

Tuyen Kieu Van: methodology, writing - review & editing;

Khanh To Van: conceptualization, investigation, methodology, formal analysis, supervision, funding acquisition, writing - original draft, review & editing.

Availability of data and materials. The experimental results are available at <https://github.com/TruongXuanHieu-H/SCL-Encoding>.

Code availability. The source code for these experiments is also available at <https://github.com/TruongXuanHieu-H/SCL-Encoding>.

Declaration

Ethics approval. Not applicable.

Consent to participate. Not applicable.

Consent for publication. Not applicable.

Conflict of interest. Not applicable.

Appendix A Correctness of the SC Encoding

In this appendix, we prove that equations (1)–(7) constitute a sound and complete encoding of the At-Most-K (AMK) constraint over decomposed blocks.

Let a block i of w variables, and define

$$S_{i,j} = \sum_{t=1}^j x_{i,t}.$$

The encoded constraint is:

$$\sum_{j=1}^w x_{i,j} \leq k.$$

The register bits $R_{i,j,s}$ are intended to represent:

$$R_{i,j,s} \equiv (S_{i,j} \geq s), \quad 1 \leq s \leq \min(j, k).$$

A.1 Soundness

Theorem 1. *Any assignment satisfying equations (1)–(7) satisfies*

$$\sum_{j=1}^w x_{i,j} \leq k.$$

Proof. Assume, for contradiction, that there exists a satisfying assignment such that

$$\sum_{j=1}^w x_{i,j} \geq k + 1.$$

Let j be the smallest index such that

$$S_{i,j} = k + 1.$$

Then necessarily

$$S_{i,j-1} = k \quad \text{and} \quad x_{i,j} = 1.$$

Since $S_{i,j-1} = k$, $R_{i,j-1,k} = 1$ (or *true*).

However, equation (7) enforces, for all $j > k$,

$$x_{i,j} \rightarrow \neg R_{i,j-1,k}.$$

Since $x_{i,j} = 1$ and $R_{i,j-1,k} = 1$, we obtain a contradiction. Therefore,

$$\sum_{j=1}^w x_{i,j} \leq k.$$

□

A.2 Completeness

Theorem 2. *If*

$$\sum_{j=1}^w x_{i,j} \leq k,$$

then there exists an assignment to the register bits $R_{i,j,s}$ satisfying equations (1)–(7).

Proof. Assume

$$\sum_{j=1}^w x_{i,j} \leq k.$$

Define the register bits as follows:

$$R_{i,j,s} := \begin{cases} 1 & \text{if } S_{i,j} \geq s, \\ 0 & \text{otherwise.} \end{cases}$$

By construction, the relation $S_{i,j} = S_{i,j-1} + x_{i,j}$ ensures that equations (1)–(3) are satisfied.

Similarly, if $S_{i,j} < s$, then either $S_{i,j-1} < s - 1$ or ($S_{i,j-1} < s$ and $x_{i,j} = 0$), which guarantees that equations (4)–(6) are satisfied.

Finally, since $S_{i,j} \leq k$ for all j , we never have both $x_{i,j} = 1$ and $S_{i,j-1} = k$ simultaneously. Hence, equation (7) is satisfied.

Therefore, a satisfying assignment of the register bits exists. □

Together, Theorems 1 and 2 establish that the *SC* encoding is sound and complete for encoding AMK constraints.

References

- [1] Artigues, C., Hebrard, E., Mayer-Eichberger, V., Siala, M., Walsh, T.: Sat and hybrid models of the car sequencing problem. In: Integration of AI and OR Techniques in Constraint Programming: 11th International Conference, CPAIOR 2014, Cork, Ireland, May 19-23, 2014. Proceedings 11, pp. 268–283 (2014). Springer
- [2] Siala, M.: Search, propagation, and learning in sequencing and scheduling problems. PhD thesis, INSA de Toulouse (2015)

- [3] Ceschia, S., Dang, N.T.T., De Causmaecker, P., Haspeslagh, S., Schaerf, A.: Second international nurse rostering competition (inrc-ii)—problem description and rules—. arXiv preprint arXiv:1501.04177 (2015)
- [4] Kletzander, L., Musliu, N.: Solving the general employee scheduling problem. *Computers & Operations Research* **113**, 104794 (2020)
- [5] Bergman, D., Cire, A.A., Hoeve, W.: Mdd propagation for sequence constraints. *Journal of Artificial Intelligence Research* **50**, 697–722 (2014)
- [6] Siala, M., Hebrard, E., Huguet, M.-J.: An optimal arc consistency algorithm for a particular case of sequence constraint. *Constraints* **19**, 30–56 (2014)
- [7] Nieuwenhuis, R., Oliveras, A., Rodríguez-Carbonell, E., Rollon, E.: Employee scheduling with sat-based pseudo-boolean constraint solving. *IEEE access* **9**, 142095–142104 (2021)
- [8] Lester, M.M.: Pseudo-boolean optimisation for robinx sports timetabling. *Journal of Scheduling* **25**(3), 287–299 (2022)
- [9] Bessiere, C., Hebrard, E., Hnich, B., Kiziltan, Z., Quimper, C.-G., Walsh, T.: Reformulating global constraints: The slide and regular constraints. In: *Abstraction, Reformulation, and Approximation: 7th International Symposium, SARA 2007, Whistler, Canada, July 18-21, 2007. Proceedings 7*, pp. 80–92 (2007). Springer
- [10] Sinnl, M.: A note on computational approaches for the antibandwidth problem. *Central European Journal of Operations Research* **29**(3), 1057–1077 (2021)
- [11] Vardi, M.: The sat revolution: Solving, sampling, and counting. In: *Mathematical Colloquium* (2015)
- [12] Haberlandt, A., Green, H., Heule, M.J.: Effective auxiliary variables via structured reencoding. arXiv preprint arXiv:2307.01904 (2023)
- [13] Vasconcellos-Gaete, C., Barichard, V., Lardeux, F.: Abacus: A new hybrid encoding for sat problems. In: *2020 IEEE 32nd International Conference on Tools with Artificial Intelligence (ICTAI)*, pp. 145–152 (2020). IEEE
- [14] Fazekas, K., Sinnl, M., Biere, A., Parragh, S.: Duplex encoding of staircase at-most-one constraints for the antibandwidth problem. In: *International Conference on Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, pp. 186–204 (2020). Springer
- [15] Sinz, C.: Towards an optimal cnf encoding of boolean cardinality constraints. In: *International Conference on Principles and Practice of Constraint Programming*, pp. 827–831 (2005). Springer

- [16] Hieu, T.X., Tuyen, K.V., Khanh, T.V.: Sequential counter encoding for staircase at-most-one constraints. In: Proceedings of the 17th International Conference on Agents and Artificial Intelligence - Volume 2: ICAART, pp. 164–175 (2025). <https://doi.org/10.5220/0013124600003890> . SciTePress
- [17] Eén, N., Sörensson, N.: Translating pseudo-boolean constraints into sat. *Journal on Satisfiability, Boolean Modelling and Computation* **2**(1-4), 1–26 (2006)
- [18] Abío, I., Nieuwenhuis, R., Oliveras, A., Rodríguez-Carbonell, E., Mayer-Eichberger, V.: A new look at bdds for pseudo-boolean constraints. *Journal of Artificial Intelligence Research* **45**, 443–480 (2012)
- [19] Asín, R., Nieuwenhuis, R., Oliveras, A., Rodríguez-Carbonell, E.: Cardinality networks and their applications. In: Theory and Applications of Satisfiability Testing-SAT 2009: 12th International Conference, SAT 2009, Swansea, UK, June 30–July 3, 2009. Proceedings 12, pp. 167–180 (2009). Springer
- [20] Asín, R., Nieuwenhuis, R., Oliveras, A., Rodríguez-Carbonell, E.: Cardinality networks: a theoretical and empirical study. *Constraints* **16**, 195–221 (2011)
- [21] Batcher, K.E.: Sorting networks and their applications. In: Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference, pp. 307–314 (1968)
- [22] Bryant, R.E.: Graph-based algorithms for boolean function manipulation. *Computers, IEEE Transactions on* **100**(8), 677–691 (1986)
- [23] Akers: Binary decision diagrams. *IEEE Transactions on computers* **100**(6), 509–516 (1978)
- [24] Chen, J.: A new sat encoding of the at-most-one constraint. *Proc. constraint modelling and reformulation*, 8 (2010)
- [25] Leung, J.Y., Vornberger, O., Witthoff, J.D.: On some variants of the bandwidth minimization problem. *SIAM Journal on Computing* **13**(3), 650–667 (1984)
- [26] Hale, W.K.: Frequency assignment: Theory and applications. *Proceedings of the IEEE* **68**(12), 1497–1514 (1980)
- [27] Cappanera, P.: A survey on obnoxious facility location problems. University of Pisa (1999)
- [28] Hu, Y., Gansner, E.R., Kobourov, S.: Visualizing graphs and clusters as maps. *IEEE Computer Graphics and Applications* **30**(6), 54–66 (2010)
- [29] Gent, I.P., Petrie, K.E., Puget, J.-F.: Symmetry in constraint programming. *Foundations of Artificial Intelligence* **2**, 329–376 (2006)
- [30] Biere, A.: Cadical at the sat race 2019. In: Heule, M., Jarvisalo, M., Suda, M.

(eds.) Proceedings of SAT Race 2019: Solver and Benchmark Descriptions, vol. B-2019-1 of Department of Computer Science Series of Publications B, University of Helsinki 2019, pp. 8–9 (2019)

- [31] Rodriguez-Tello, E., Romero-Monsivais, H., Ramírez-Torres, J., Lardeux, F.: Harwell-Boeing Graphs for the CB Problem. <https://doi.org/10.13140/2.1.2408.0166>
- [32] Nethercote, N., Stuckey, P.J., Becket, R., Brand, S., Duck, G.J., Tack, G.: Minizinc: Towards a standard cp modelling language. In: International Conference on Principles and Practice of Constraint Programming, pp. 529–543 (2007). Springer